

Bayesian Learning - Computer Lab 3

Liuxi Mei

Xiaochen Liu

2025-0-13

1. Gibbs sampling for the logistic regression

Consider again the logistic regression model in problem 2 from the previous computer lab 2. Use the prior $\beta \sim \mathcal{N}(0, \tau^2 I)$, where $\tau = 3$.

(a)

Implement (code!) a Gibbs sampler that simulates from the joint posterior $p(\omega, \beta | x)$ by augmenting the data with Polya-gamma latent variables $\omega_i, i = 1, \dots, n$. The full conditional posteriors are given on the slides from Lecture 7. Evaluate the convergence of the Gibbs sampler by calculating the Inefficiency Factors (IFs) and by plotting the trajectories of the sampled Markov chains.

```
library(BayesLogit)
library(mvtnorm)

data = read.csv('Disease.csv')
x <- as.matrix(data[, 1:6])
y <- as.matrix(data[, 7])
# initialize beta
nDraws = 1000
burnin <- 200      # Burn-in period

n <- nrow(x)
p <- ncol(x)

beta <- rep(1, 6) # 5+1 intercept

beta_samples <- matrix(NA, nDraws, p)

set.seed(123)

for (i in 1:nDraws) {
  omega <- rpg(n, 1, x %*% beta)
  k <- y - 1 / 2
  B <- 9 * diag(6)
  b <- rep(0, p)
  # Conditional posterior for beta

  v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
  m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)
```

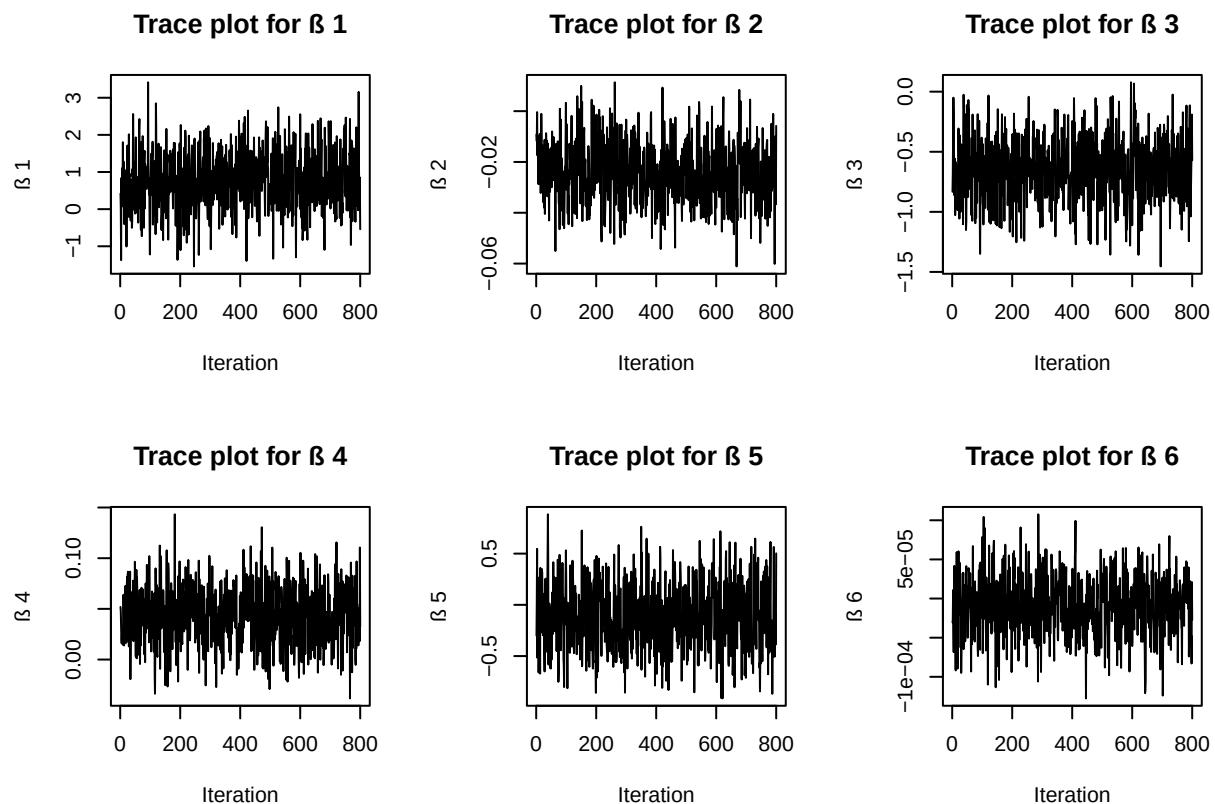
```

beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
beta_samples[i, ] <- beta
}
# Discard burn-in
beta_samples <- beta_samples[(burnin + 1):nDraws, ]
nPost <- nrow(beta_samples)

par(mfrow = c(2, 3)) # 6 pics

for (i in 1:p) {
  plot(
    beta_samples[, i],
    type = 'l',
    main = paste("Trace plot for ", i),
    xlab = "Iteration",
    ylab = paste(" ", i)
  )
}

```



```

max_lag <- 50 # set the max lag
acfs <- matrix(NA, max_lag, p)

for (i in 1:p) {
  chain <- beta_samples[, i] - mean(beta_samples[, i])
  var_chain <- mean(chain ^ 2)
}

```

```

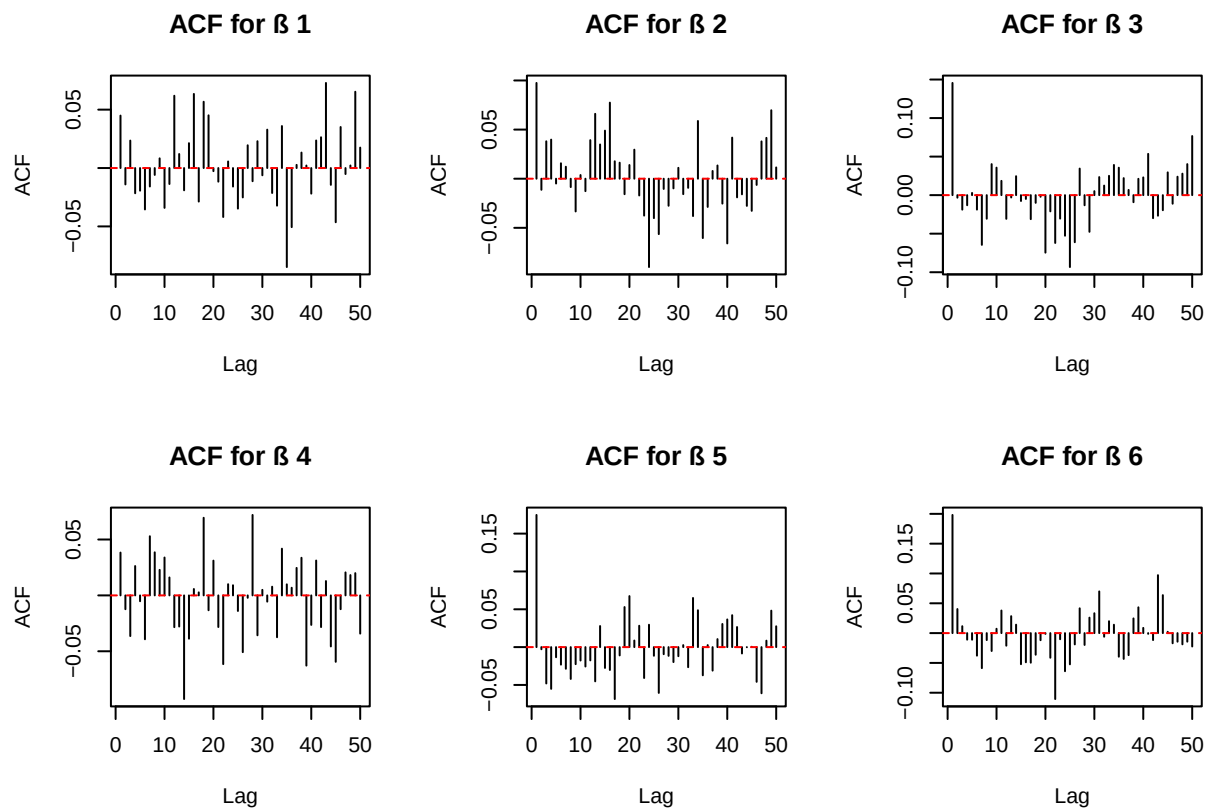
for (j in 1:max_lag) { # j as lag
  # calculate the autocorrelation factor
  acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / nPost / var_chain
}

}

par(mfrow = c(2, 3)) # 6 pics

for (i in 1:p) {
  plot(
    1:max_lag,
    acfs[, i],
    type = 'h',
    main = paste("ACF for ", i),
    xlab = "Lag",
    ylab = "ACF"
  )
  abline(h = 0, col = "red", lty = 2)
}

```



```

# inefficiency factor (IF)
IFs <- numeric(p)

```

```

for (i in 1:p) {
  IFs[i] <- 1 + 2 * sum(acfs[, i])
  cat(paste("IF for ", i, "=", round(IFs[i], 2), "\n"))
}

```

```

## IF for 1 = 1.16
## IF for 2 = 1.25
## IF for 3 = 0.95
## IF for 4 = 0.72
## IF for 5 = 0.77
## IF for 6 = 0.72

```

Plots show that all efficient fluctuate around certain levels without any obvious tendency or any stuck. And calculated inefficiency factors are around 1, which means very sampling are performed with quite low dependency and the efficiency is very good.

(b)

Repeat the same task as in (a), but now only use the first m observations of the dataset. Do this for all $m \in \{10, 40, 80\}$.

```

m <- c(10, 40, 80)
for (z in 1:3) {
  row_nr <- m[z]
  data = read.csv('Disease.csv')
  x <- as.matrix(data [1:row_nr, 1:6])
  y <- as.matrix(data [1:row_nr, 7])

  # initialize beta
  nDraws = 1000
  burnin <- 200      # Burn-in period

  n <- nrow(x)
  p <- ncol(x)

  beta <- rep(1, 6) # 5+1 intercept

  beta_samples <- matrix(NA, nDraws, p)

  set.seed(123)

  for (i in 1:nDraws) {
    omega <- rpg(n, 1, x %*% beta)
    k <- y - 1 / 2
    B <- 9 * diag(6)
    b <- rep(0, p)
    # Conditional posterior for beta

    v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
    m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)
  }
}

```

```

    beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
    beta_samples[i, ] <- beta
  }
  # Discard burn-in
  beta_samples <- beta_samples[(burnin + 1):nDraws, ]
  nPost <- nrow(beta_samples)

  par(mfrow = c(2, 3)) # 6 pics

  for (i in 1:p) {
    plot(
      beta_samples[, i],
      type = 'l',
      main = paste("Trace plot for ", i , "with first ", row_nr),
      xlab = "Iteration",
      ylab = paste(" ", i)
    )
  }

  max_lag <- 50 # set the max lag
  acfs <- matrix(NA, max_lag, p)

  for (i in 1:p) {
    chain <- beta_samples[, i] - mean(beta_samples[, i])
    var_chain <- mean(chain ^ 2)
    for (j in 1:max_lag) { # j as lag
      # calculate the autocorrelation factor
      acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / nPost / var_chain
    }
  }

  par(mfrow = c(2, 3)) # 6 pics

  for (i in 1:p) {
    plot(
      1:max_lag,
      acfs[, i],
      type = 'h',
      main = paste("ACF for ", i , "with first ", row_nr),
      xlab = "Lag",
      ylab = "ACF"
    )
    abline(h = 0, col = "red", lty = 2)
  }

  # inefficiency factor (IF)
  IFs <- numeric(p)

  for (i in 1:p) {
    IFs[i] <- 1 + 2 * sum(acfs[, i])
  }

```

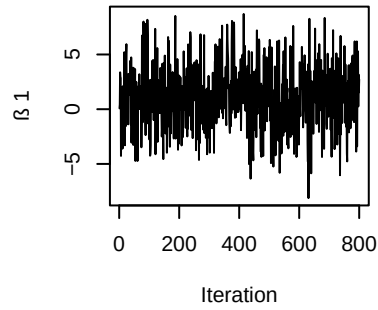
```

cat(paste("IF for ", i, "=", round(IFs[i], 2), "with first ", row_nr, "\n"))
}

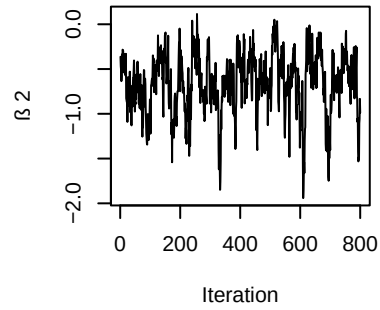
}

```

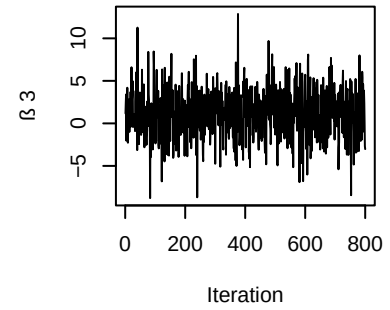
Trace plot for β_1 with first 10



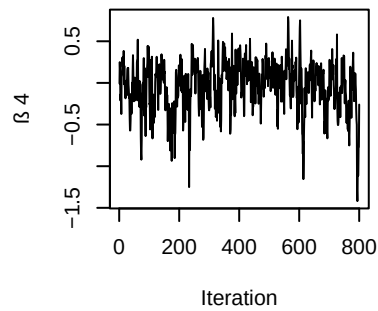
Trace plot for β_2 with first 10



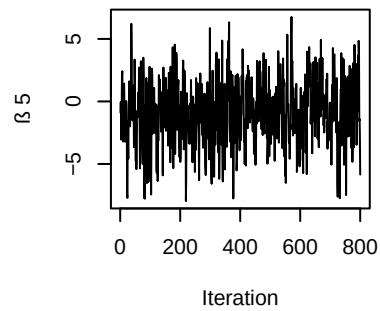
Trace plot for β_3 with first 10



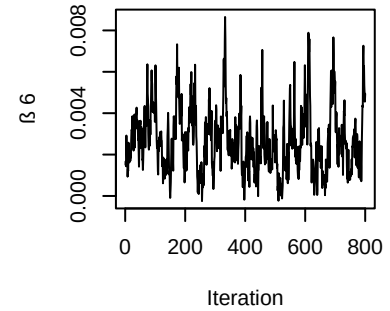
Trace plot for β_4 with first 10



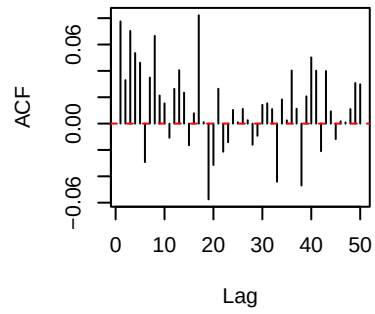
Trace plot for β_5 with first 10



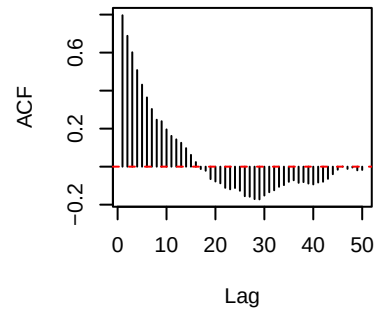
Trace plot for β_6 with first 10



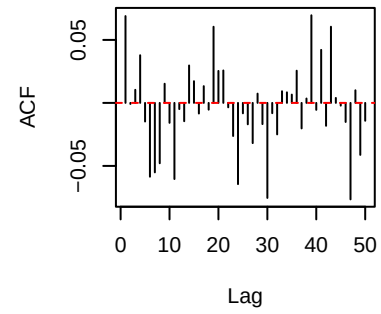
ACF for β_1 with first 10



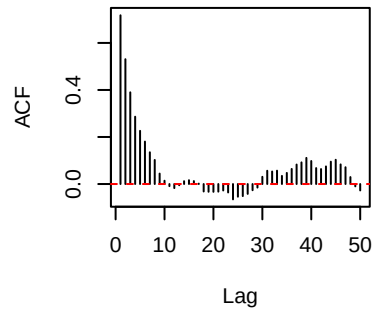
ACF for β_2 with first 10



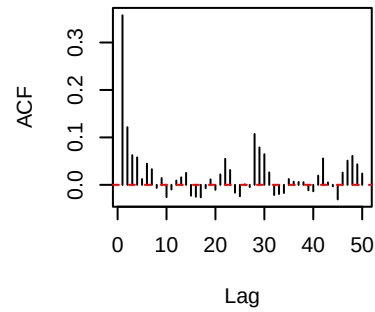
ACF for β_3 with first 10



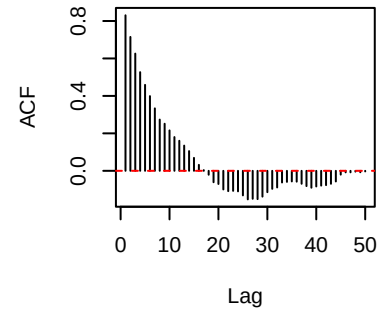
ACF for β_4 with first 10



ACF for β_5 with first 10

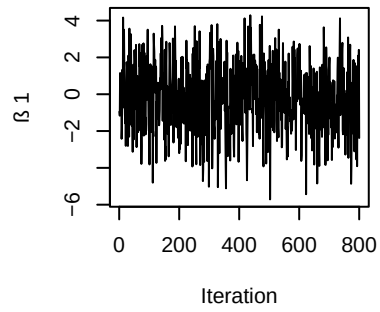


ACF for β_6 with first 10

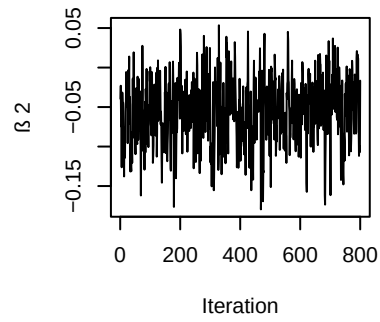


```
## IF for 1 = 2.34 with first 10
## IF for 2 = 5.3 with first 10
## IF for 3 = 0.59 with first 10
## IF for 4 = 7.94 with first 10
## IF for 5 = 3.34 with first 10
## IF for 6 = 6.65 with first 10
```

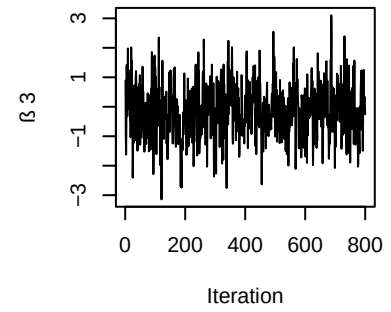
Trace plot for β_1 with first 40



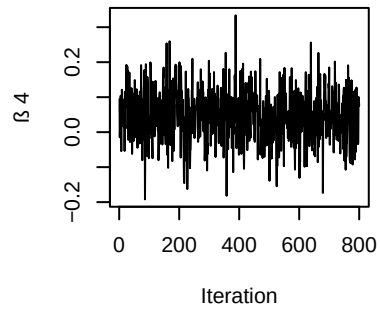
Trace plot for β_2 with first 40



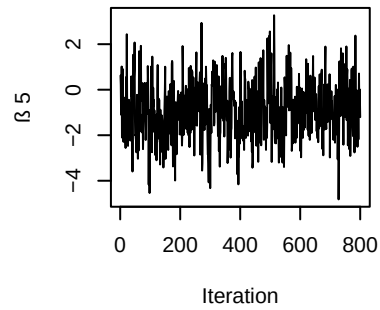
Trace plot for β_3 with first 40



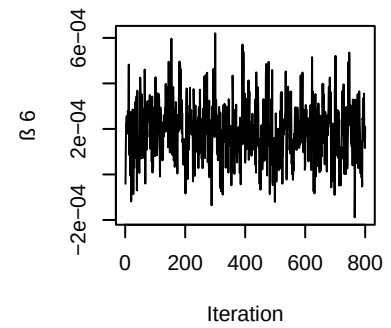
Trace plot for β_4 with first 40



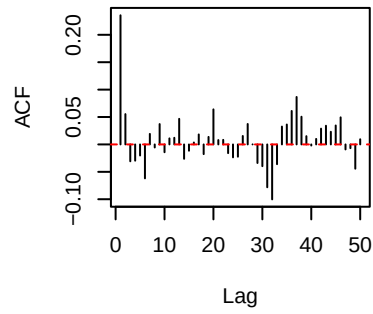
Trace plot for β_5 with first 40



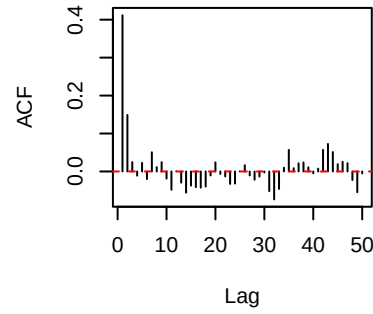
Trace plot for β_6 with first 40



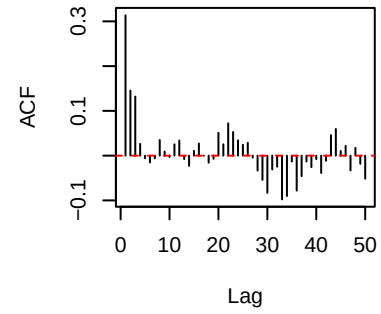
ACF for β_1 with first 40



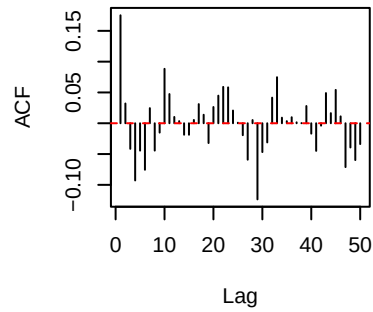
ACF for β_2 with first 40



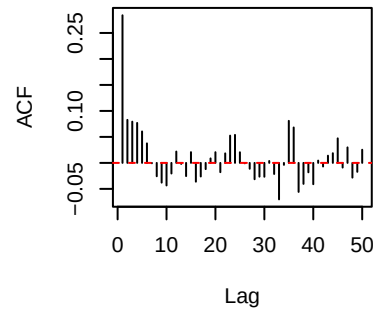
ACF for β_3 with first 40



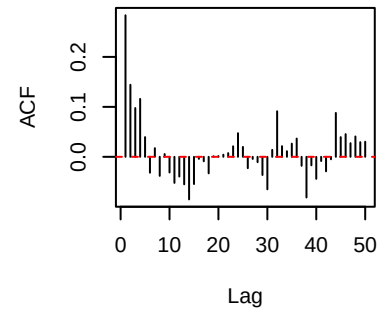
ACF for β_4 with first 40



ACF for β_5 with first 40

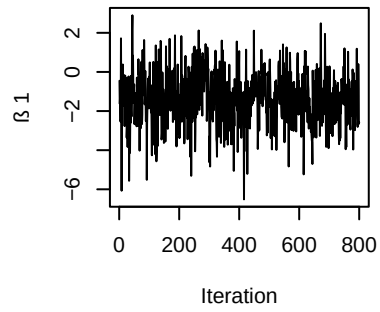


ACF for β_6 with first 40

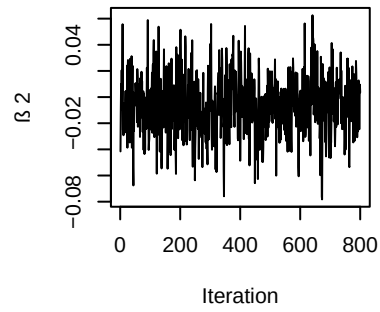


```
## IF for 1 = 1.85 with first 40
## IF for 2 = 1.75 with first 40
## IF for 3 = 1.73 with first 40
## IF for 4 = 1.02 with first 40
## IF for 5 = 1.95 with first 40
## IF for 6 = 2.06 with first 40
```

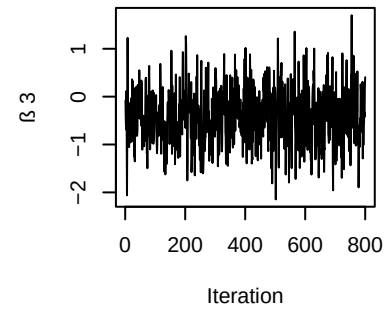
Trace plot for β_1 with first 80



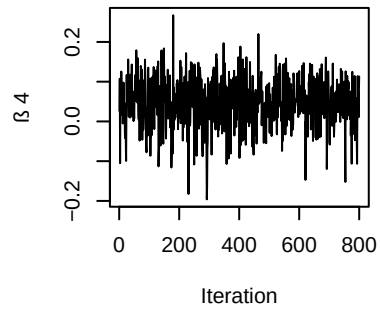
Trace plot for β_2 with first 80



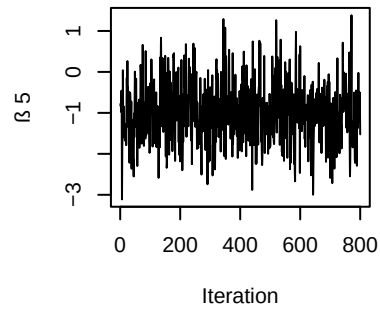
Trace plot for β_3 with first 80



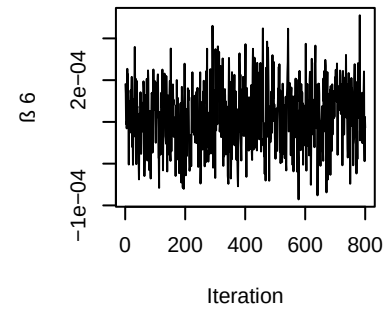
Trace plot for β_4 with first 80

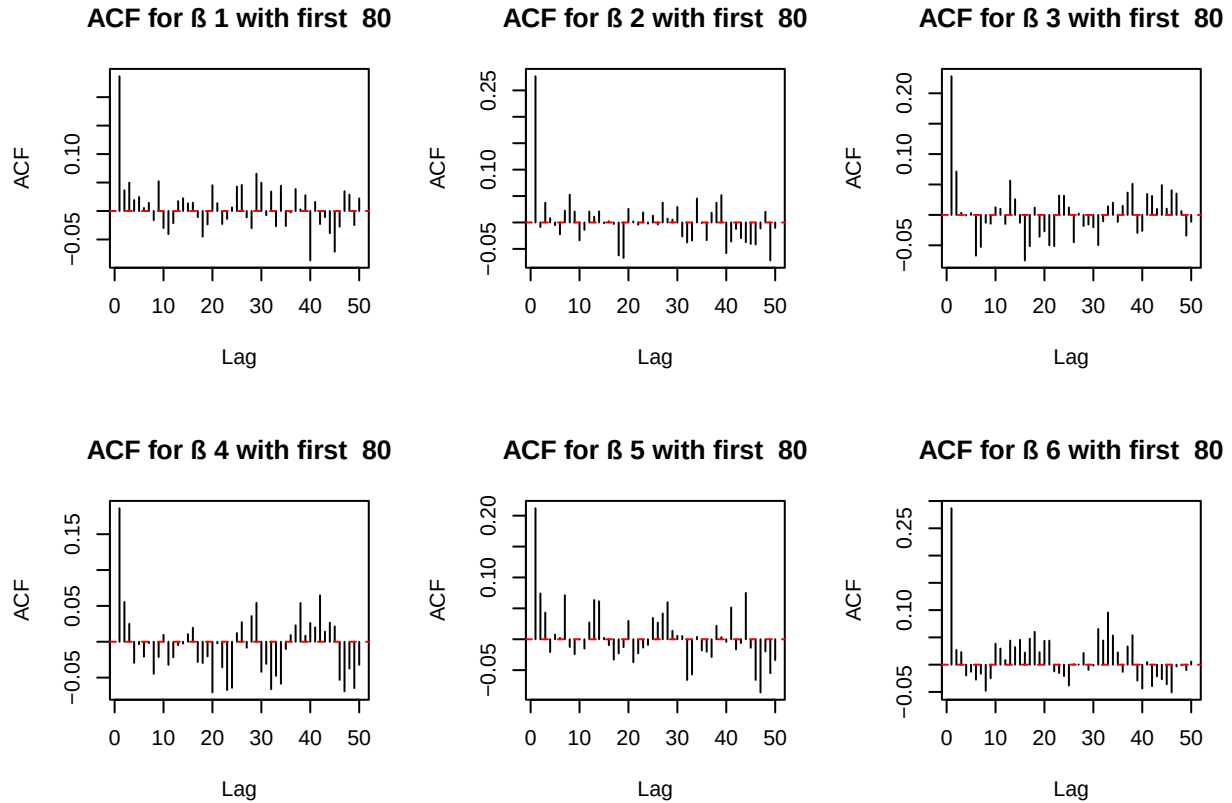


Trace plot for β_5 with first 80



Trace plot for β_6 with first 80





```
## IF for 1 = 1.83 with first 80
## IF for 2 = 1.15 with first 80
## IF for 3 = 1.23 with first 80
## IF for 4 = 0.36 with first 80
## IF for 5 = 1.42 with first 80
## IF for 6 = 2.31 with first 80
```

As for case 1 where we have 10 observations, there is a decreasing tendency noticed for β_4 and the overall IF is comparatively high, thus it is not considered to converge.

For case 2 and 3 where there are 40 and 80 observations respectively, all plots have reached a stabilization and all parameters have a good IF factor. Thus convergence is thought to be reached.

2. Metropolis Random Walk for Poisson regression

Consider the following Poisson regression model:

$$y_i | \beta \sim \text{Poisson}(\exp(x_i^T \beta)), \quad i = 1, \dots, n,$$

where y_i is the count for the i th observation in the sample and x_i is the p -dimensional vector with covariate observations for the i th observation.

The dataset contains observations from 700 eBay auctions of coins. The response variable is `nBids` and records the number of bids in each auction. The remaining variables are features/covariates (x):

- **Const:** Intercept
- **PowerSeller:** Equal to 1 if the seller is selling large volumes on eBay
- **VerifyID:** Equal to 1 if the seller is a verified seller by eBay
- **Sealed:** Equal to 1 if the coin was sold in an unopened envelope
- **MinBlem:** Equal to 1 if the coin has a minor defect
- **MajBlem:** Equal to 1 if the coin has a major defect
- **LargNeg:** Equal to 1 if the seller received a lot of negative feedback from customers
- **LogBook:** Logarithm of the book value of the auctioned coin according to expert sellers (Standardized)
- **MinBidShare:** Ratio of the minimum selling price (starting price) to the book value (Standardized).

(a)

Obtain the maximum likelihood estimator of β in the Poisson regression model for the eBay data. [Hint: glm.R, don't forget that glm() adds its own intercept so don't input the covariate Const.] Which covariates are significant?

```
rm(list=ls())

data <- read.table('eBayNumberOfBidderData_2025.dat', header = FALSE, skip = 1)

colnames(data) <- c("nBids", "Const", "PowerSeller", "VerifyID", "Sealed", "MinBlem", "MajBlem", "LargNeg", "LogBook", "MinBidShare")

# Fit Poisson regression model (excluding Const)

model <- glm(nBids ~ . - Const, family = poisson, data = data)
summary(model)
```

```
##
## Call:
## glm(formula = nBids ~ . - Const, family = poisson, data = data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.08534    0.03592  30.213 < 2e-16 ***
## PowerSeller -0.02833    0.04433  -0.639  0.52280
## VerifyID    -0.34902    0.13008  -2.683  0.00729 **
## Sealed      0.50101    0.06676   7.504 6.18e-14 ***
## MinBlem     -0.12189    0.08388  -1.453  0.14619
## MajBlem     -0.24884    0.09865  -2.523  0.01165 *
## LargNeg      0.03610    0.07075   0.510  0.60987
## LogBook     -0.07280    0.03505  -2.077  0.03783 *
## MinBidShare -1.76665    0.08292 -21.306 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 1407.17  on 699  degrees of freedom
## Residual deviance:  593.76  on 691  degrees of freedom
## AIC: 2529.5
##
## Number of Fisher Scoring iterations: 5
```

Lower P value means higher significance, as is shown, significant covariants are VerifyID and MinBidShare.

(b)

Let's do a Bayesian analysis of the Poisson regression. Let the prior be $\beta \sim \mathcal{N}(0, 100 \cdot (X^\top X)^{-1})$, where X is the $n \times p$ covariate matrix. This is a commonly used prior, which is called Zellner's g-prior. Assume first that the posterior density is approximately multivariate normal:

$$\beta \mid y \sim \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where $\tilde{\beta}$ is the posterior mode and $J_y(\tilde{\beta})$ is the negative Hessian at the posterior mode.

$\tilde{\beta}$ and $J_y(\tilde{\beta})$ can be obtained by numerical optimization (`optim.R`) exactly like you already did for the logistic regression in Lab 2 (but with the log posterior function replaced by the corresponding one for the Poisson model, which you have to code up).

```
y<- matrix(data$nBids, ncol=1)
x<- as.matrix(data[, 2:10])

LogPostPoisson <- function(beta,y,X){
  beta <- as.numeric(beta)
  lin_pred <- X %*% beta
  p = length(beta)

  # Poisson log-likelihood
  log_lik <- sum(y * lin_pred - exp(lin_pred))

  # Zellner's g-prior: beta ~ N(0, 100 * (X^T X)^{-1})
  log_prior = dmvnorm(beta, rep(0, p), 100*solve(t(X)%*%X), log=TRUE);
  return(log_lik + log_prior)
}

OptimRes <- optim(par = rep(0, ncol(x)),
  fn = LogPostPoisson,
  X = x, y = y,
  method = "BFGS",
  control = list(fnscale = -1, maxit = 1000),
  hessian = TRUE)
approxPostMode <- matrix(OptimRes$par,1,9)
cov_matrix <- solve(-OptimRes$hessian)# J^{-1}
```

(c)

Let's simulate from the actual posterior of β using the Metropolis algorithm and compare the results with the approximate results in (b). Program a general function that uses the Metropolis algorithm to generate random draws from an arbitrary posterior density. In order to show that it is a general function for any model, we denote the vector of model parameters by θ . Let the proposal density be the multivariate normal density mentioned in Lecture 8 (random walk Metropolis):

$$\theta^{(p)} \mid \theta^{(i-1)} \sim \mathcal{N}(\theta^{(i-1)}, c \cdot \Sigma),$$

where $\Sigma = J_y^{-1}(\tilde{\beta})$ was obtained in (b). The value c is a tuning parameter and should be an input to your Metropolis function. The user of your Metropolis function should be able to supply her own posterior density function, not necessarily for the Poisson regression, and still be able to use your Metropolis function. This is not so straightforward, unless you have come across function objects in R. The note `HowToCodeRWM.pdf` in Lisam describes how you can do this in R.

Now, use your new Metropolis function to sample from the posterior of β in the Poisson regression for the eBay dataset. Assess MCMC convergence by graphical methods.

```
mp_sim <- function(c, LogPostFunc, ndraws, theta_init, sigma) {
  p <- length(theta_init)
  theta_mat <- matrix(NA, ndraws, p)
  theta_mat[1, ] <- theta_init

  for (i in 2:ndraws) {
    theta_new <- rmvnorm(1, theta_mat[i - 1, ], c * sigma)

    log_post_new <- LogPostFunc(theta_new, y = y, X = x)
    log_post_old <- LogPostFunc(theta_mat[i - 1, ], y = y, X = x)

    acc_prob <- min(1, exp(log_post_new - log_post_old))

    if (acc_prob > runif(1)) {
      theta_mat[i, ] <- theta_new
    } else {
      theta_mat[i, ] <- theta_mat[i - 1, ]
    }
  }

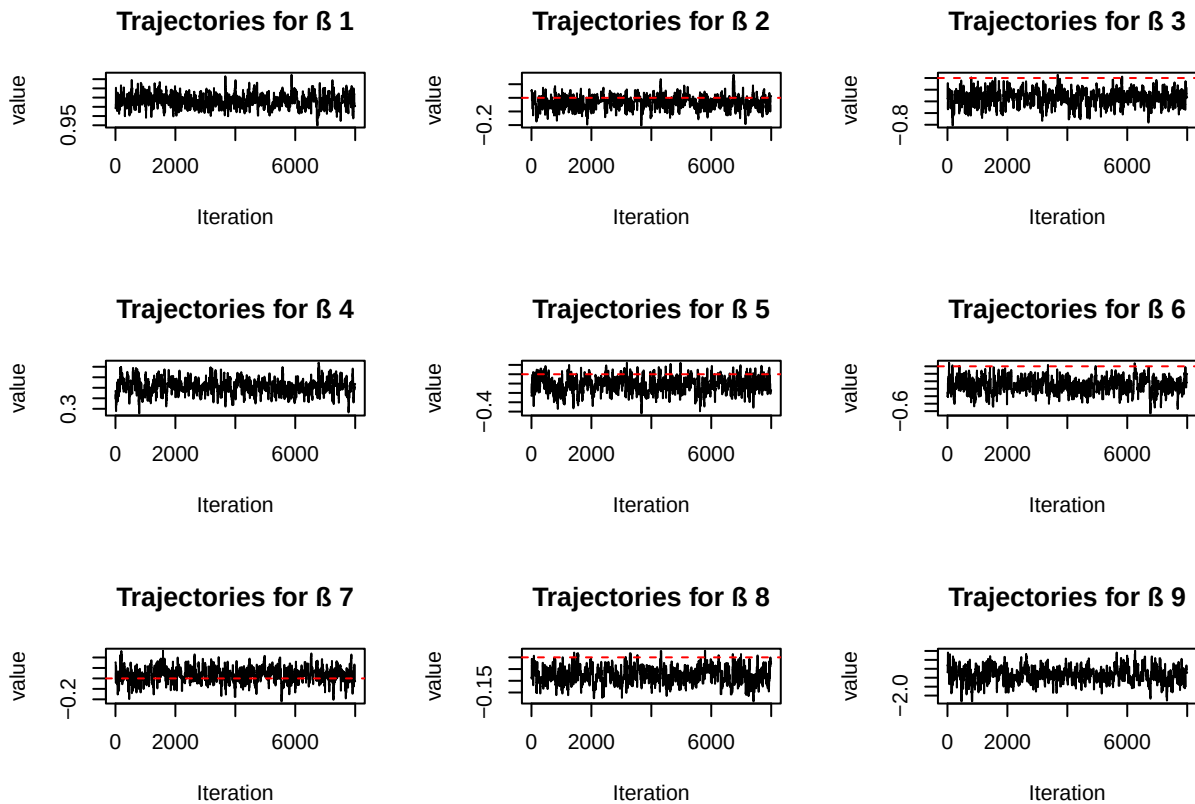
  return(theta_mat)
}
```

```
set.seed(123)
poission_samp <- mp_sim(1, LogPostFunc = LogPostPoisson,
                        ndraws=10000,
                        theta_init=OptimRes$par,
                        sigma=cov_matrix)
```

```
par(mfrow = c(3, 3)) # 9 pics
p <- length(OptimRes$par)
burnin <- 2000

for (i in 1:p) {
  plot(
    1:(dim(poission_samp)[1]-burnin),
    poission_samp[(burnin+1):nrow(poission_samp), i],
    type = 'l',
    main = paste("Trajectories for ", i),
    xlab = "Iteration",
    ylab = "value"
  )
}
```

```
abline(h = 0, col = "red", lty = 2)
}
```



```
ndraws <- nrow(possion_samp)
draw_plot <- ndraws-burnin
for (j in 1:9) {
  param_index <- j # choose the ith param

  mean_vec <- numeric(draw_plot)
  sd_vec <- numeric(draw_plot)

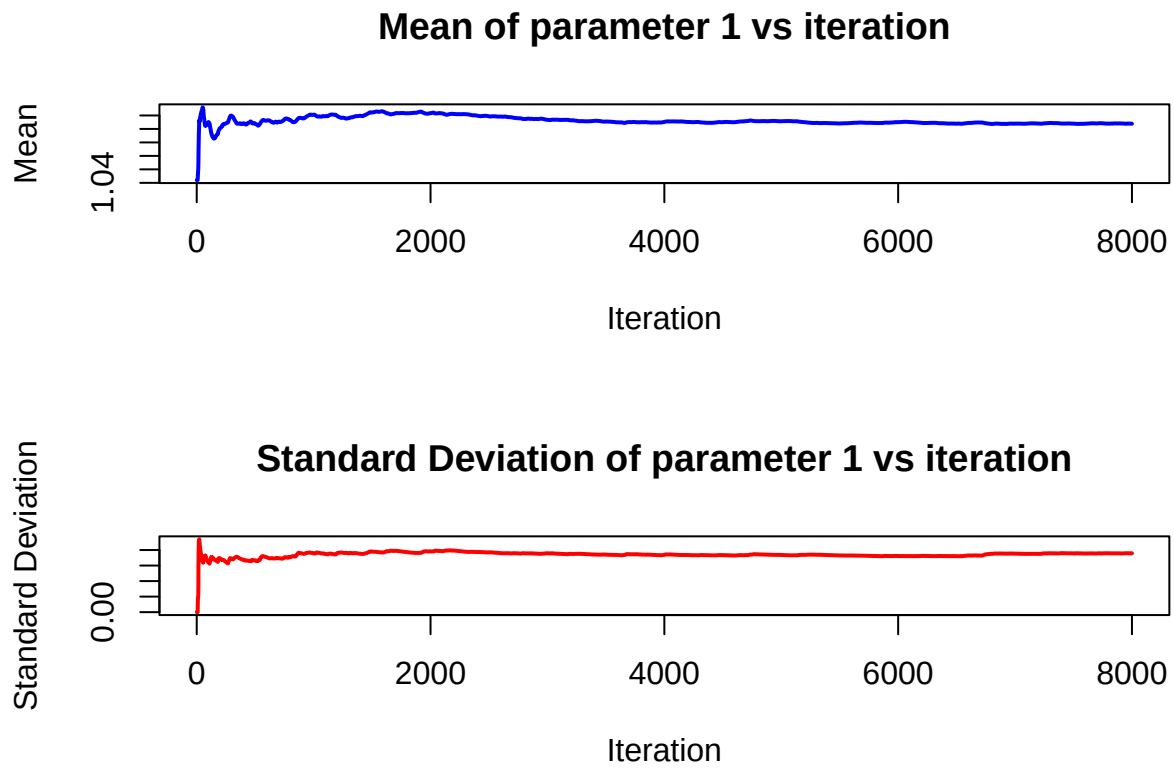
  for (i in 1:draw_plot) {
    samples_so_far <- possion_samp[(burnin+1):(i+burnin),param_index]
    mean_vec[i] <- mean(samples_so_far)
    sd_vec[i] <- sd(samples_so_far)
  }

  par(mfrow=c(2,1))

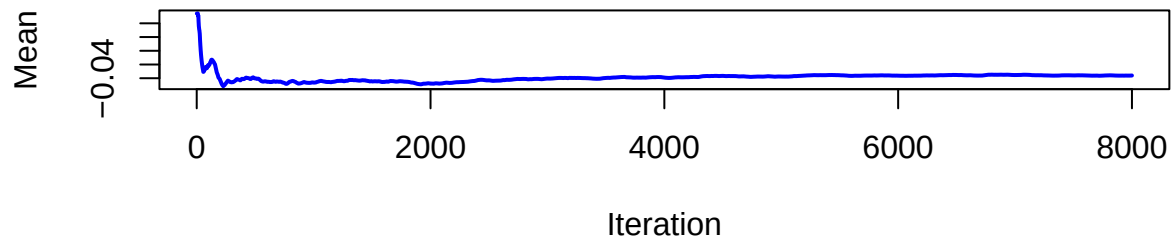
  plot(1:draw_plot, mean_vec, type='l', col='blue', lwd=2,
       xlab='Iteration', ylab='Mean',
       main=paste('Mean of parameter', param_index, 'vs iteration'))

  plot(1:draw_plot, sd_vec, type='l', col='red', lwd=2,
```

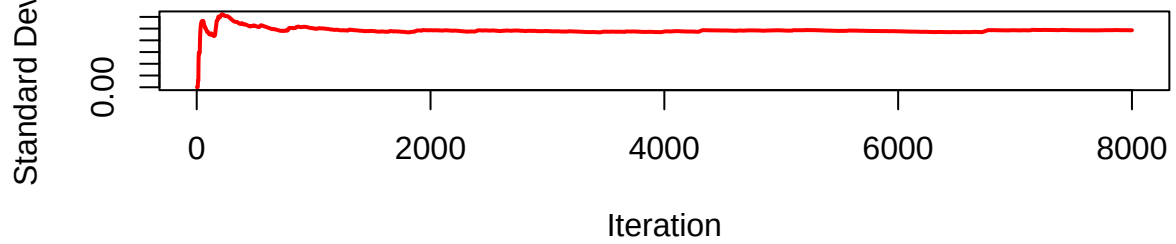
```
xlab='Iteration', ylab='Standard Deviation',  
main=paste('Standard Deviation of parameter', param_index, 'vs iteration'))  
}
```



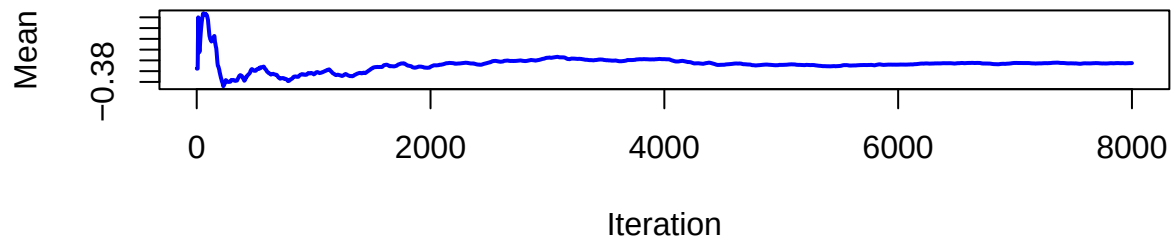
Mean of parameter 2 vs iteration



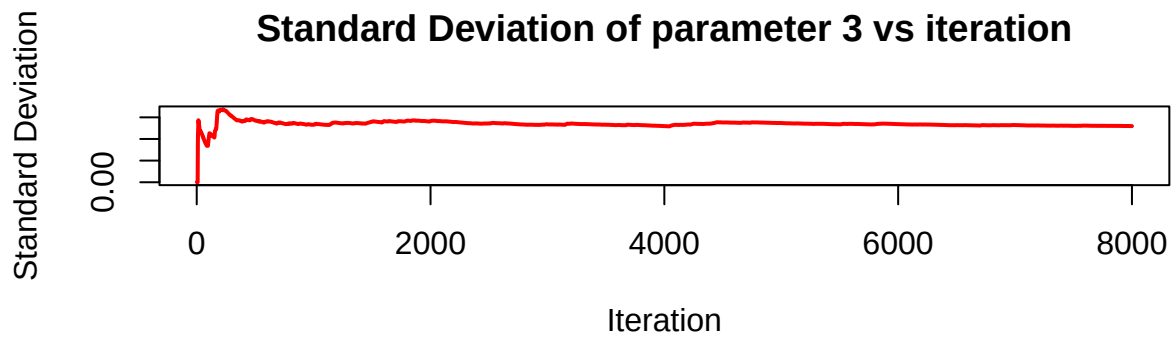
Standard Deviation of parameter 2 vs iteration



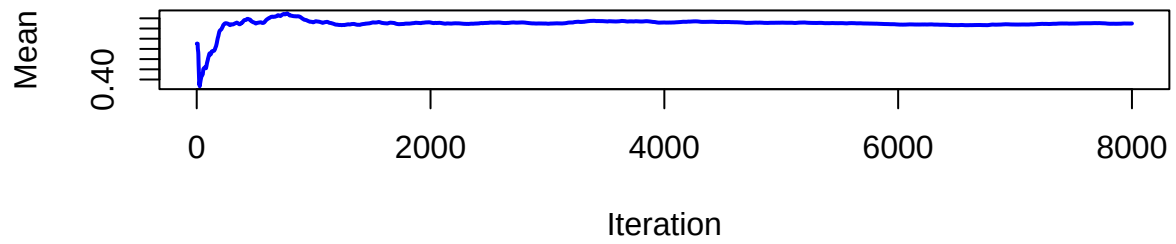
Mean of parameter 3 vs iteration



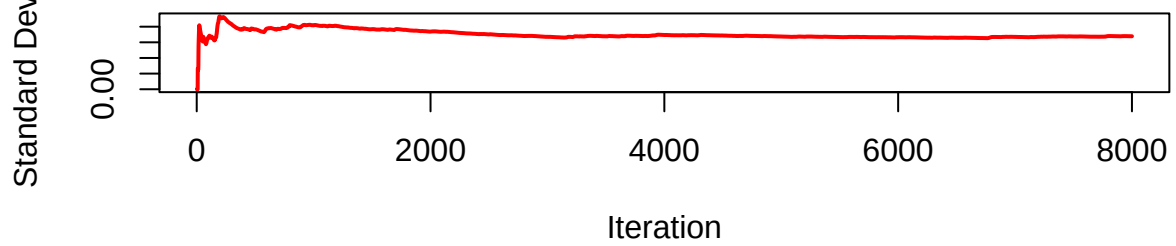
Standard Deviation of parameter 3 vs iteration



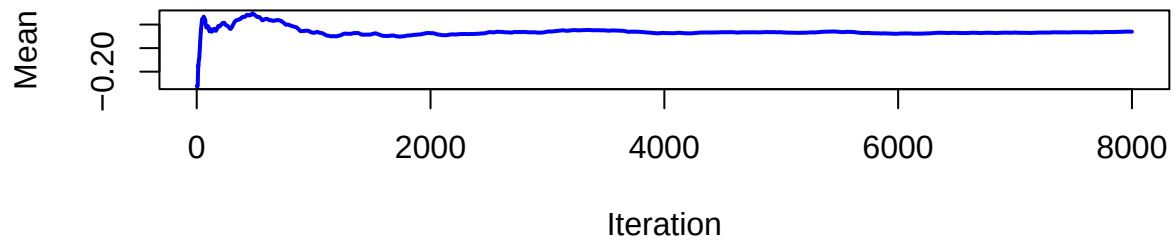
Mean of parameter 4 vs iteration



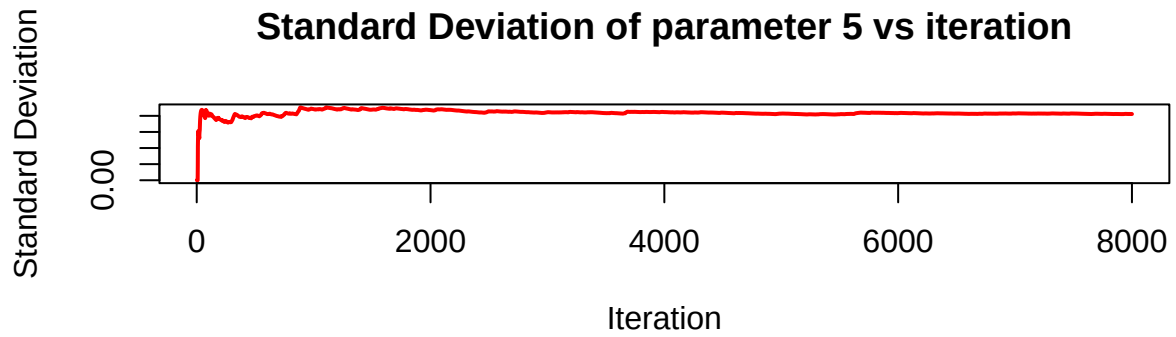
Standard Deviation of parameter 4 vs iteration



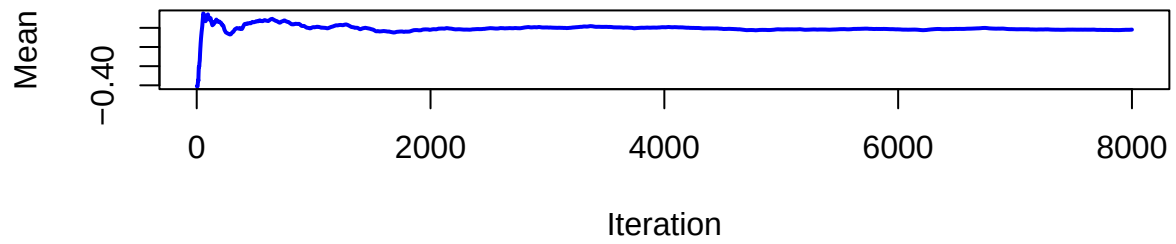
Mean of parameter 5 vs iteration



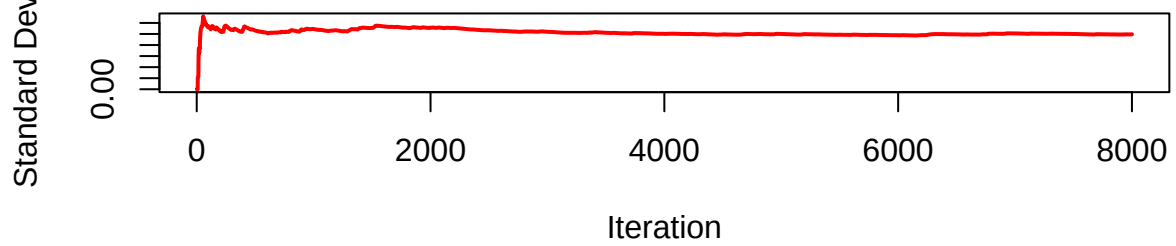
Standard Deviation of parameter 5 vs iteration



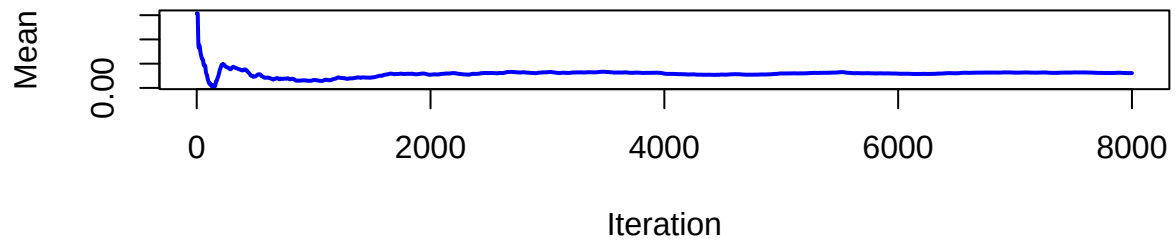
Mean of parameter 6 vs iteration



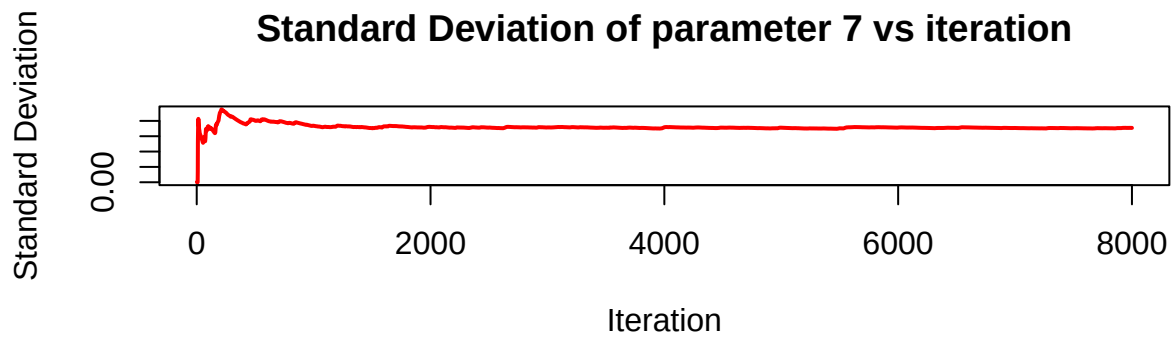
Standard Deviation of parameter 6 vs iteration



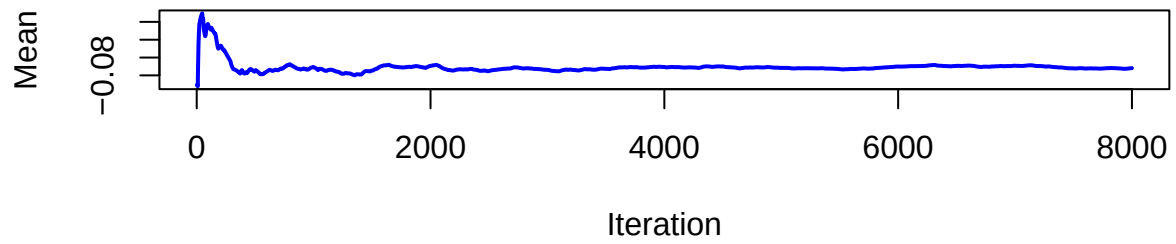
Mean of parameter 7 vs iteration



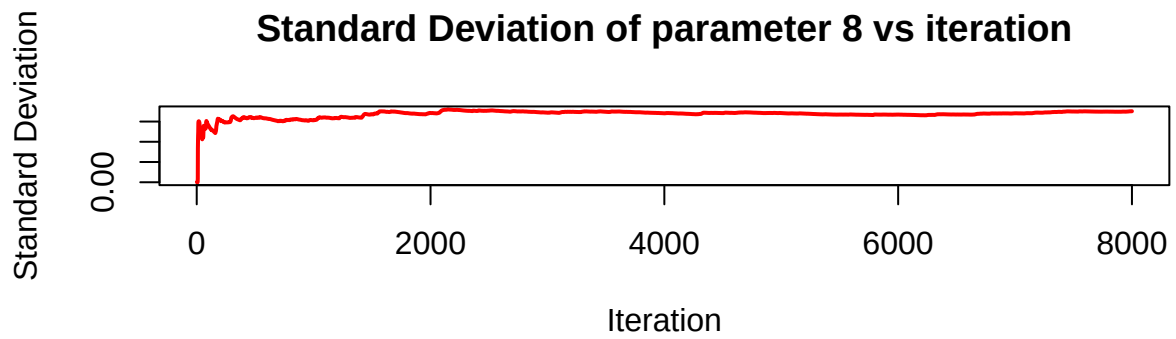
Standard Deviation of parameter 7 vs iteration



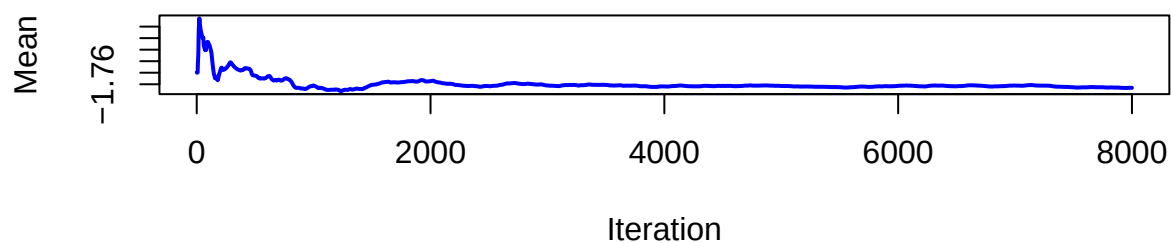
Mean of parameter 8 vs iteration



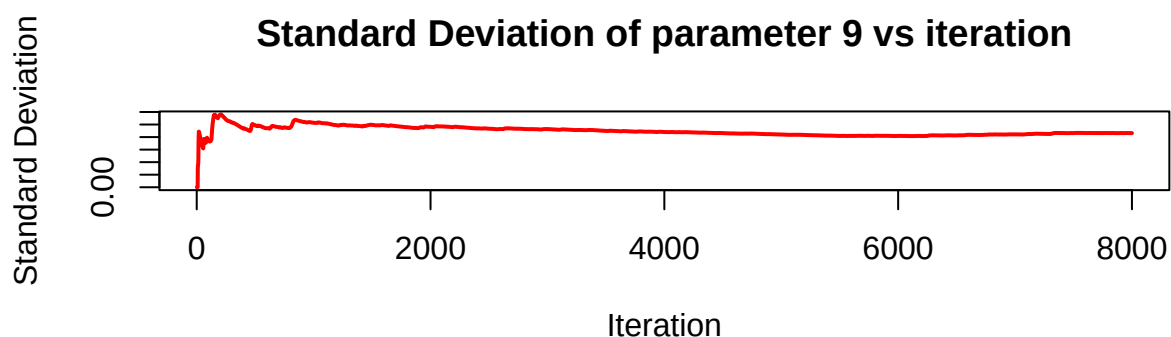
Standard Deviation of parameter 8 vs iteration



Mean of parameter 9 vs iteration



Standard Deviation of parameter 9 vs iteration



Since trajectories and the mean and standard deviation for different parameters are stable, convergence is thus considered reached.

(d)

Use the MCMC draws from (c) to simulate from the predictive distribution of the number of bidders in a new auction with the characteristics below. Plot the predictive distribution. What is the probability of no bidders in this new auction?

- PowerSeller = 1
- VerifyID = 0
- Sealed = 1
- MinBlem = 0
- MajBlem = 1
- LargNeg = 0
- LogBook = 1.3

- $\text{MinBidShare} = 0.7$

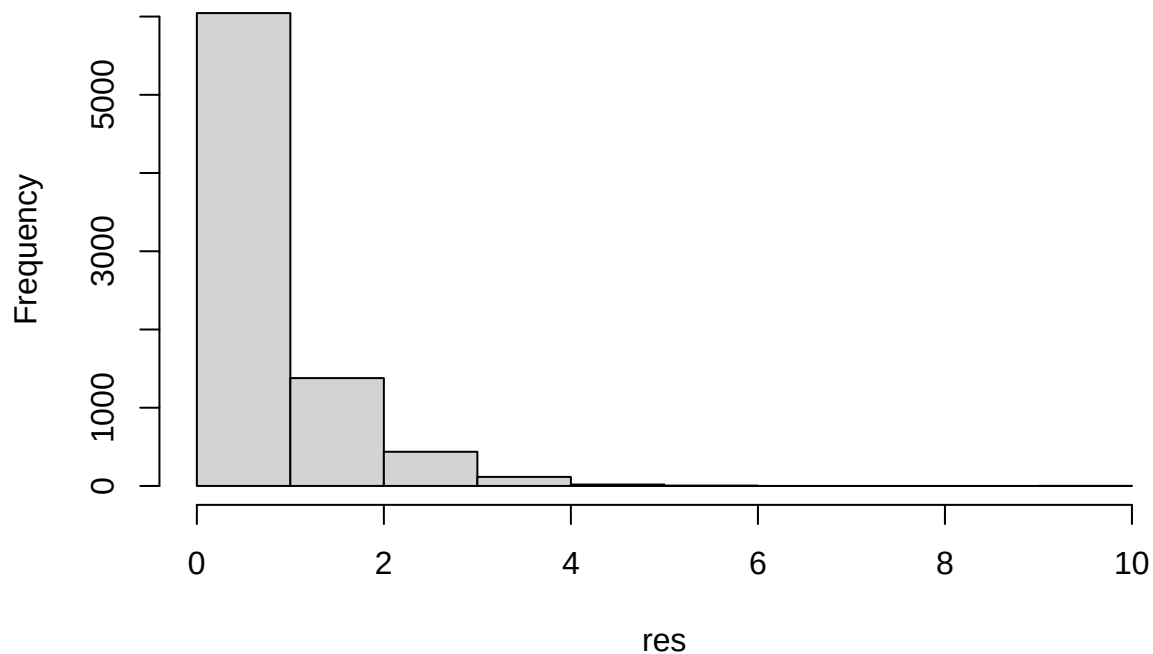
```
bidden <- c(1,1,0,1,0,1,0,1.3,0.7) # intercept as 1 to include it
lambda <- exp(possion_samp[(burnin+1):ndraws,] %*% bidden)

res <- rep(NA, length(lambda))

for (i in 1:length(lambda)) {
  res[i] <- rpois(1, lambda[i])
}

hist(res)
```

Histogram of res



```
prob_0 <- sum(res == 0)/length(lambda)
print(prob_0)
```

```
## [1] 0.387625
```

3. Time series models in Stan

(a)

Write a function in R that simulates data from the AR(1)-process

$$x_t = \mu + \phi(x_{t-1} - \mu) + \varepsilon_t, \quad \varepsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, \sigma^2),$$

for given values of μ , ϕ and σ^2 . Start the process at $x_1 = \mu$ and then simulate values for x_t for $t = 2, 3, \dots, T$ and return the vector $x_{1:T}$ containing all time points. Use $\mu = 5$, $\sigma^2 = 9$, and $T = 300$ and look at some different realizations (simulations) of $x_{1:T}$ for values of ϕ between -1 and 1 (this is the interval of ϕ where the AR(1)-process is stationary). Include a plot of at least one realization in the report. What effect does the value of ϕ have on $x_{1:T}$?

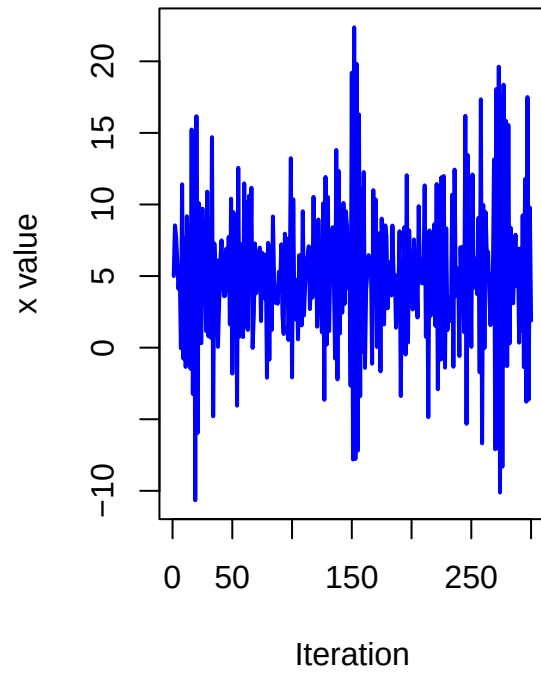
```
rm(list=ls())
#define the function

ar <- function (mu, phi, sig2, T) {
  res<- c()
  x<- mu
  res<-x
  for (t in 2:T) {
    x_new <- mu + phi*(x-mu)+rnorm(1,0,sqrt(sig2))
    res<- c(res,x_new)
    x<- x_new
  }
  return(res)
}

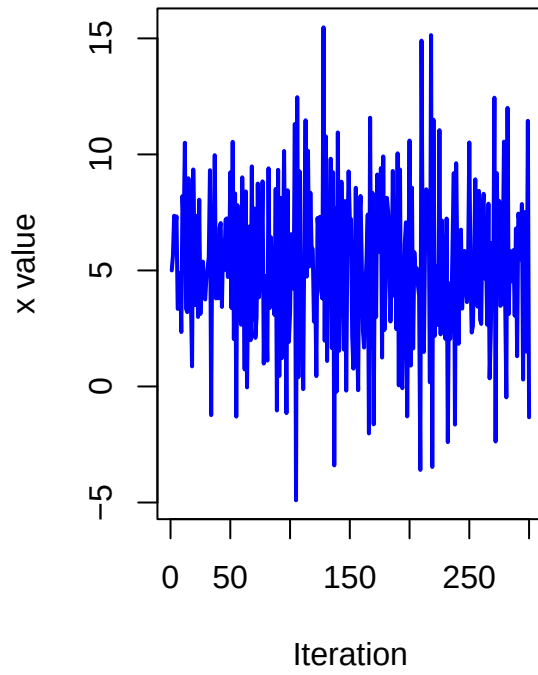
#simulate
set.seed(123)
p <- seq(-1,1,0.1)
res <- matrix(NA,nrow=300, ncol=length(p))
for (i in 1:length(p)) {
  res[,i] <-ar (mu=5, phi=p[i], sig2=9, T=300)
}
```

```
nplot <- seq(3,21,3)
par(mfrow = c(1, 2)) # 6 pics
for (i in nplot) {
  plot(res[,i], type='l', col='blue', lwd=2,
        xlab='Iteration', ylab='x value',
        main=paste('x changes with',p[i], 'as phi'))
}
```

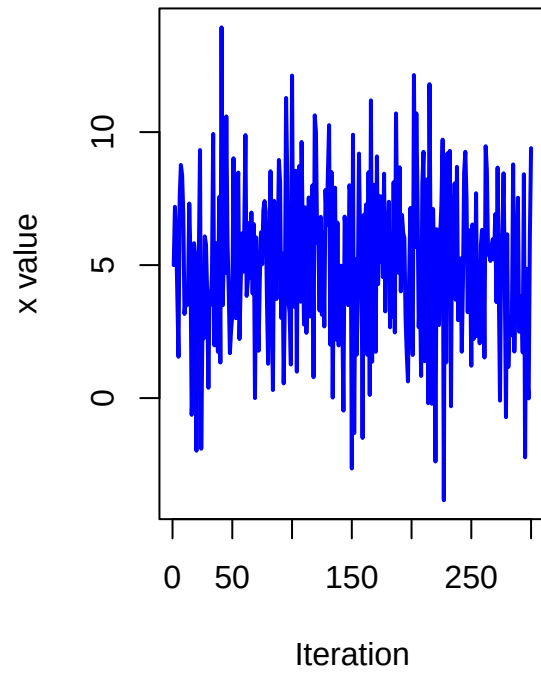
x changes with -0.8 as ϕ



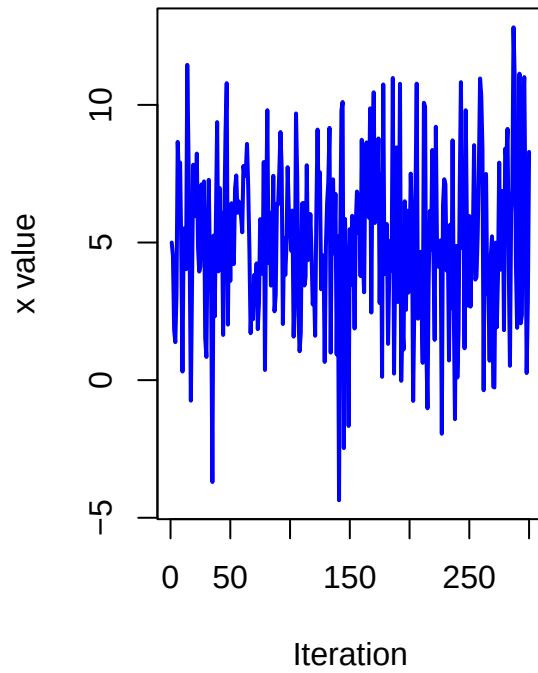
x changes with -0.5 as ϕ



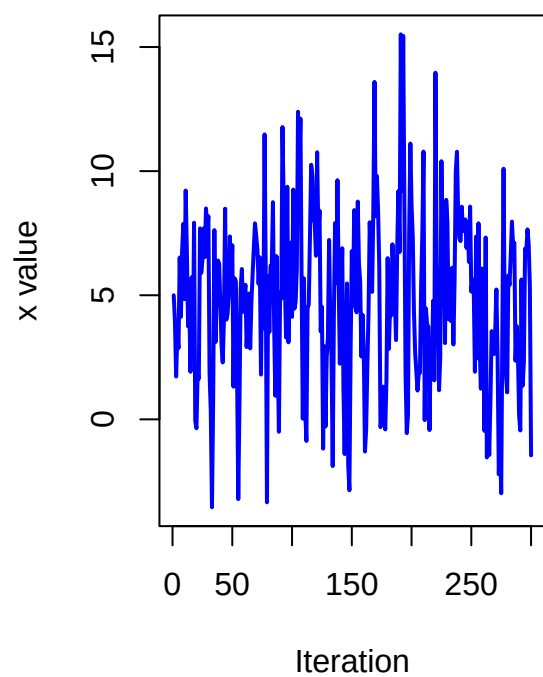
x changes with -0.2 as ϕ



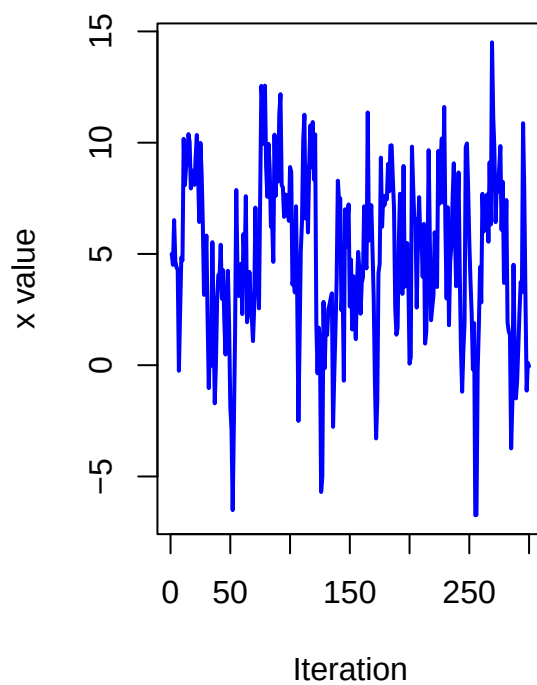
x changes with 0.1 as ϕ



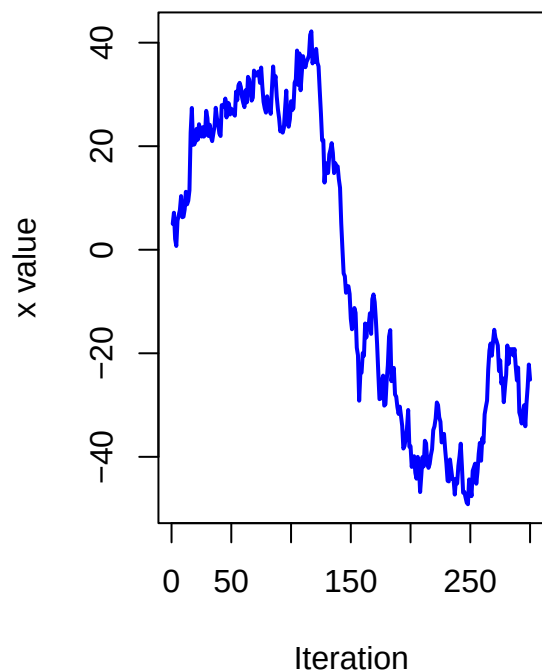
x changes with 0.4 as phi



x changes with 0.7 as phi



x changes with 1 as phi



ϕ controls the strength of autocorrelation. ϕ with small absolute value gives weak correlation and values fluctuate quickly.

(b)

Use your function from (a) to simulate two AR(1)-processes, $x_{1:T}$ with $\phi = 0.4$ and $y_{1:T}$ with $\phi = 0.98$. Now, treat your simulated vectors as synthetic data, and treat the values of μ , ϕ and σ^2 as unknown parameters. Implement Stan-code that samples from the posterior of the three parameters, using suitable non-informative priors of your choice.

Hint: Look at the time-series models examples in the Stan user's guide/reference manual, and note the different parameterization used here.

- i. Report the posterior mean, 95% credible intervals, and the number of effective posterior samples for the three inferred parameters for each of the simulated AR(1)-processes. Are you able to estimate the true values?
- ii. For each of the two data sets, evaluate the convergence of the samplers and plot the joint posterior of μ and ϕ . Comments?

```
set.seed(123)
ar1 <- ar(mu=5, phi=0.4, sig2=9, T=300)
ar2 <- ar(mu=5, phi=0.98, sig2=9, T=300)
```

```

library(rstan)

##      StanHeaders

##
## rstan version 2.32.7 (Stan version 2.32.2)

## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)

## Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file

y=ar1
N=length(y)

StanModel = '
data {
  int<lower=0> N; // Number of observations
  vector[N] y; // Number of flowers
}
parameters {
  real mu;
  real<lower=-1, upper=1> phi;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 5);
  phi ~ uniform(-1,1);
  sigma ~ normal(3, 3); // Normal with mean 3, st.dev. 3  It is normal (mean, sd_dev) in stan

  for(i in 2:N){
    y[i]~ normal(mu+ phi*(y[i-1] - mu), sigma);
  }
}'

data <- list(N=N, y=y)
warmup <- 50
niter <- 2000
fit <- stan(model_code=StanModel,data=data, warmup=warmup,iter=niter,chains=4)

## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.

```

```
## Trying to compile a simple C file
```

```
## Running "C:/PROGRA~1/R/R-44~1.3/bin/x64/Rcmd.exe" SHLIB foo.c
## using C compiler: 'gcc.exe (GCC) 13.3.0'
## gcc -I"C:/PROGRA~1/R/R-44~1.3/include" -DNDEBUG -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/Rcpp/include/" -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include."
I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/unsupported" -
I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/BH/include" -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/StanHeaders/include/src/" -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/StanHeaders/include/" -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppParallel
RCPP_PARALLEL_USE_TBB=1 -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/rstan/include" -
DEIGEN_NO_DEBUG -DBOOST_DISABLE_ASSERTS -DBOOST_PENDING_INTEGER_LOG2_HPP -DSTAN_THREADS -
DUSE_STANC3 -DSTRICT_R_HEADERS -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION -D_HAS_AUTO_PTR_ETC=0 -
include "C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan/math/prim/fun/
std=c++1y -I"C:/rtools44/x86_64-w64-mingw32.static.posix/include" -O2 -Wall -
mfpmath=sse -msse2 -mstackrealign -c foo.c -o foo.o
## cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
## In file included from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen,
## from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen,
## from C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan
## from <command-line>:
## C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/src/Core/util/Macros.
## 679 | #include <cmath>
## | ~~~~~
## compilation terminated.
## make: *** [C:/PROGRA~1/R/R-44~1.3/etc/x64/Makeconf:289: foo.o] Error 1
```

```
## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.
```

```
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 5.5e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.55 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: WARNING: There aren't enough warmup iterations to fit the
## Chain 1: three stages of adaptation as currently configured.
## Chain 1: Reducing each adaptation stage to 15%/75%/10% of
## Chain 1: the given number of warmup iterations:
## Chain 1: init_buffer = 7
## Chain 1: adapt_window = 38
## Chain 1: term_buffer = 5
## Chain 1:
## Chain 1: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 1: Iteration: 51 / 2000 [ 2%] (Sampling)
## Chain 1: Iteration: 250 / 2000 [ 12%] (Sampling)
## Chain 1: Iteration: 450 / 2000 [ 22%] (Sampling)
## Chain 1: Iteration: 650 / 2000 [ 32%] (Sampling)
```



```

## Chain 1: Iteration: 850 / 2000 [ 42%] (Sampling)
## Chain 1: Iteration: 1050 / 2000 [ 52%] (Sampling)
## Chain 1: Iteration: 1250 / 2000 [ 62%] (Sampling)
## Chain 1: Iteration: 1450 / 2000 [ 72%] (Sampling)
## Chain 1: Iteration: 1650 / 2000 [ 82%] (Sampling)
## Chain 1: Iteration: 1850 / 2000 [ 92%] (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.029 seconds (Warm-up)
## Chain 1: 0.568 seconds (Sampling)
## Chain 1: 0.597 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 2.8e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.28 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: WARNING: There aren't enough warmup iterations to fit the
## Chain 2: three stages of adaptation as currently configured.
## Chain 2: Reducing each adaptation stage to 15%/75%/10% of
## Chain 2: the given number of warmup iterations:
## Chain 2: init_buffer = 7
## Chain 2: adapt_window = 38
## Chain 2: term_buffer = 5
## Chain 2:
## Chain 2: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration: 51 / 2000 [ 2%] (Sampling)
## Chain 2: Iteration: 250 / 2000 [ 12%] (Sampling)
## Chain 2: Iteration: 450 / 2000 [ 22%] (Sampling)
## Chain 2: Iteration: 650 / 2000 [ 32%] (Sampling)
## Chain 2: Iteration: 850 / 2000 [ 42%] (Sampling)
## Chain 2: Iteration: 1050 / 2000 [ 52%] (Sampling)
## Chain 2: Iteration: 1250 / 2000 [ 62%] (Sampling)
## Chain 2: Iteration: 1450 / 2000 [ 72%] (Sampling)
## Chain 2: Iteration: 1650 / 2000 [ 82%] (Sampling)
## Chain 2: Iteration: 1850 / 2000 [ 92%] (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.03 seconds (Warm-up)
## Chain 2: 0.649 seconds (Sampling)
## Chain 2: 0.679 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 3e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.3 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: WARNING: There aren't enough warmup iterations to fit the

```

```

## Chain 3:      three stages of adaptation as currently configured.
## Chain 3:      Reducing each adaptation stage to 15%/75%/10% of
## Chain 3:      the given number of warmup iterations:
## Chain 3:      init_buffer = 7
## Chain 3:      adapt_window = 38
## Chain 3:      term_buffer = 5
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration:   51 / 2000 [ 2%] (Sampling)
## Chain 3: Iteration:  250 / 2000 [12%] (Sampling)
## Chain 3: Iteration:  450 / 2000 [22%] (Sampling)
## Chain 3: Iteration:  650 / 2000 [32%] (Sampling)
## Chain 3: Iteration:  850 / 2000 [42%] (Sampling)
## Chain 3: Iteration: 1050 / 2000 [52%] (Sampling)
## Chain 3: Iteration: 1250 / 2000 [62%] (Sampling)
## Chain 3: Iteration: 1450 / 2000 [72%] (Sampling)
## Chain 3: Iteration: 1650 / 2000 [82%] (Sampling)
## Chain 3: Iteration: 1850 / 2000 [92%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.017 seconds (Warm-up)
## Chain 3:      0.671 seconds (Sampling)
## Chain 3:      0.688 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 2.8e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.28 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: WARNING: There aren't enough warmup iterations to fit the
## Chain 4:      three stages of adaptation as currently configured.
## Chain 4:      Reducing each adaptation stage to 15%/75%/10% of
## Chain 4:      the given number of warmup iterations:
## Chain 4:      init_buffer = 7
## Chain 4:      adapt_window = 38
## Chain 4:      term_buffer = 5
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration:   51 / 2000 [ 2%] (Sampling)
## Chain 4: Iteration:  250 / 2000 [12%] (Sampling)
## Chain 4: Iteration:  450 / 2000 [22%] (Sampling)
## Chain 4: Iteration:  650 / 2000 [32%] (Sampling)
## Chain 4: Iteration:  850 / 2000 [42%] (Sampling)
## Chain 4: Iteration: 1050 / 2000 [52%] (Sampling)
## Chain 4: Iteration: 1250 / 2000 [62%] (Sampling)
## Chain 4: Iteration: 1450 / 2000 [72%] (Sampling)
## Chain 4: Iteration: 1650 / 2000 [82%] (Sampling)
## Chain 4: Iteration: 1850 / 2000 [92%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.031 seconds (Warm-up)

```

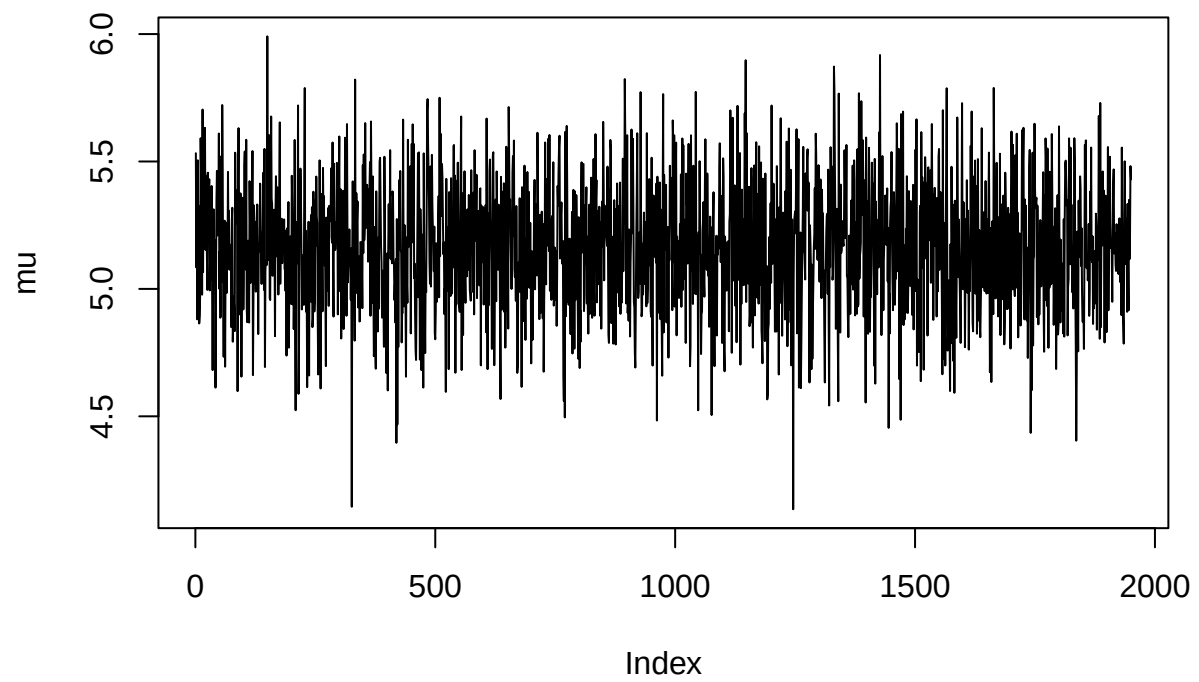
```
## Chain 4:          0.683 seconds (Sampling)
## Chain 4:          0.714 seconds (Total)
## Chain 4:
```

```
# Print the fitted model
print(fit,digits_summary=3)
```

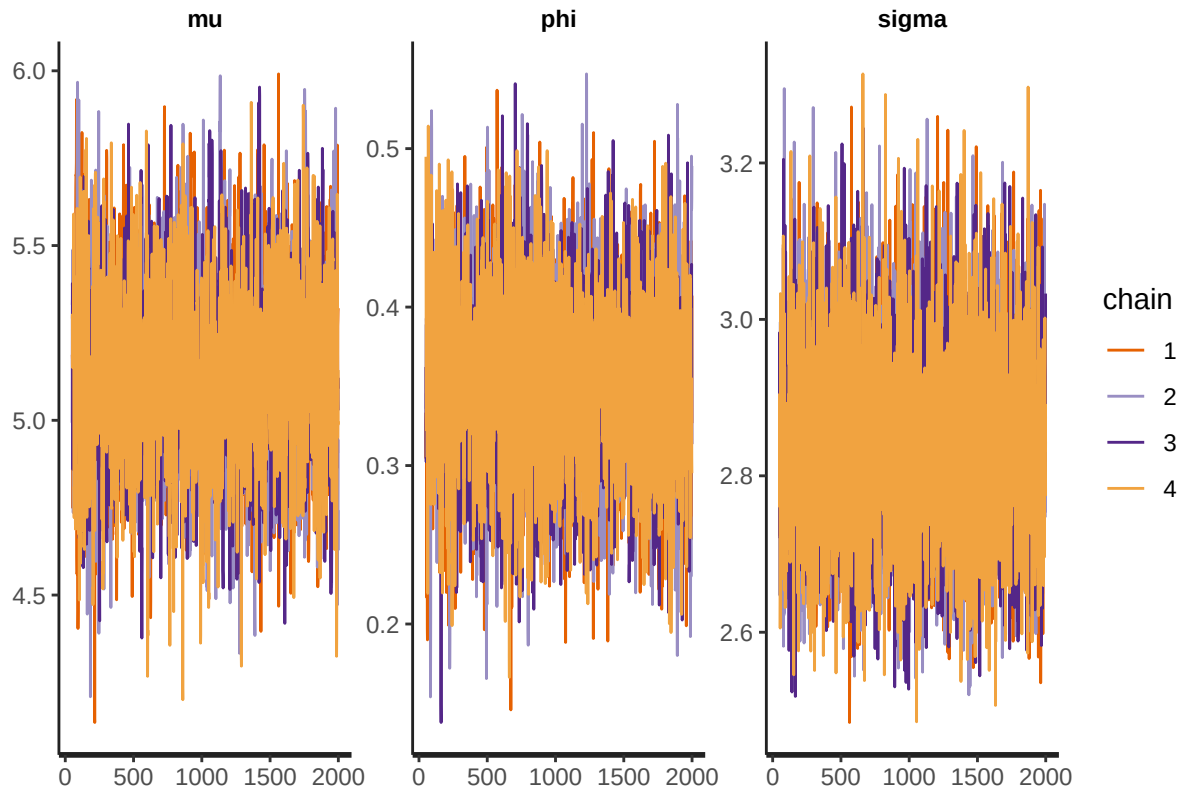
```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=50; thin=1;
## post-warmup draws per chain=1950, total post-warmup draws=7800.
##
##          mean se_mean   sd    2.5%    25%    50%    75%    97.5% n_eff
## mu       5.144   0.006 0.257   4.639   4.971   5.146   5.318   5.640  2007
## phi      0.351   0.001 0.054   0.246   0.315   0.351   0.388   0.459  4177
## sigma    2.844   0.001 0.117   2.628   2.764   2.841   2.919   3.088  8620
## lp__    -461.836  0.025 1.248 -465.031 -462.386 -461.506 -460.920 -460.436 2493
##          Rhat
## mu       1.002
## phi      1.001
## sigma    1.000
## lp__     1.001
##
## Samples were drawn using NUTS(diag_e) at Fri May 16 23:43:51 2025.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
# Extract posterior samples
postDraws <- extract(fit)
# Do traceplots of the first chain
par(mfrow = c(1,1))
plot(postDraws$mu[1:(niter-warmup)],type="l",ylab="mu",main="Traceplot")
```

Traceplot

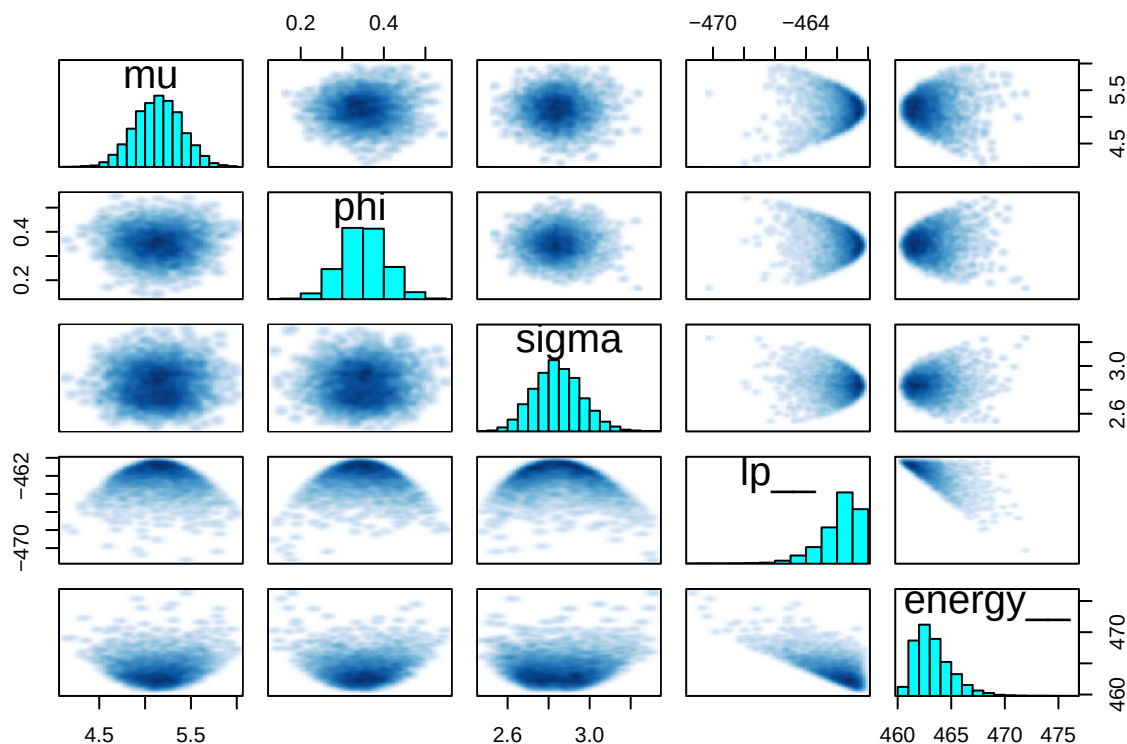


```
# Do automatic traceplots of all chains  
traceplot(fit)
```



```
# Bivariate posterior plots
pairs(fit)
```

```
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
```



```
summary_fit <- summary(fit, pars = c("mu", "phi", "sigma"))$summary
# extract the mean, 95% CI and effective posterior sample
posterior_summary <- summary_fit[, c("mean", "2.5%", "97.5%", "n_eff")]
print(round(posterior_summary, 3))
```

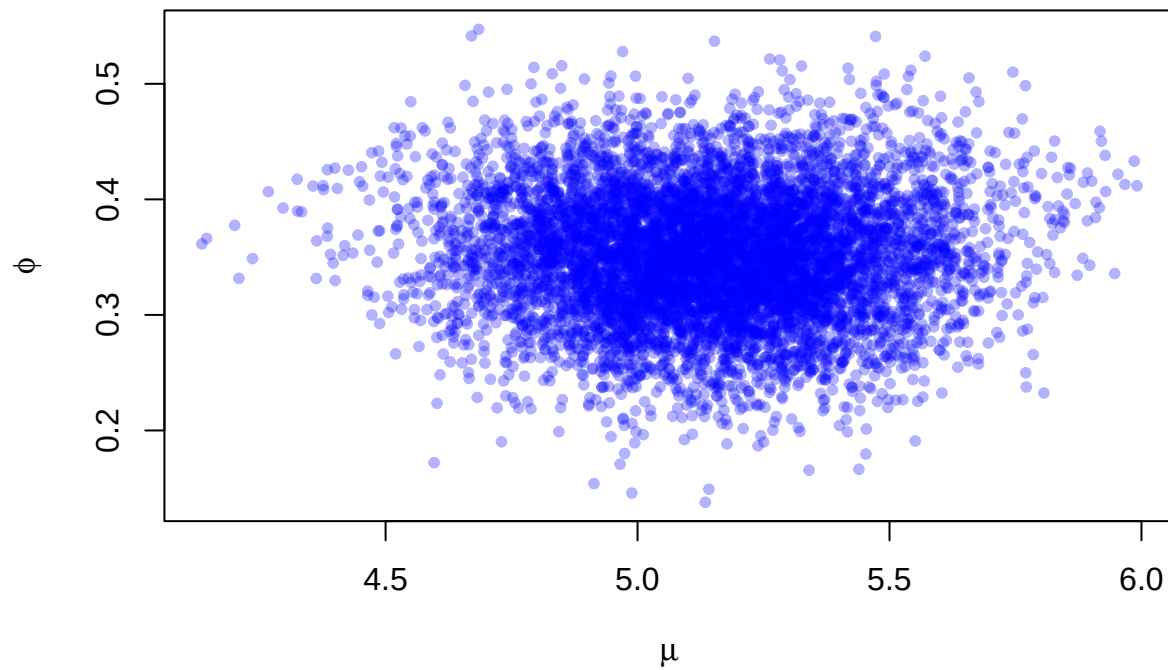
```
##      mean 2.5% 97.5%  n_eff
## mu    5.144 4.639 5.640 2006.820
## phi   0.351 0.246 0.459 4176.580
## sigma 2.844 2.628 3.088 8619.746
```

Mean, 95% CI and effective posterior sample number are shown as above. The simulated parameter values are quite close to the true parameter values and the true parameter lies in the 95% CI of simulated parameter values, thus we are able to estimate the true values.

```
posterior_samples <- as.data.frame(rstan::extract(fit, pars = c("mu", "phi")))

plot(posterior_samples$mu, posterior_samples$phi,
     xlab = expression(mu), ylab = expression(phi),
     main = "Joint Posterior of  and ", pch = 20, col = rgb(0, 0, 1, 0.3))
```

Joint Posterior of μ and ϕ



```
y=ar2
N=length(y)

StanModel = '
data {
  int<lower=0> N; // Number of observations
  vector[N] y; // Number of flowers
}
parameters {
  real mu;
  real<lower=-1, upper=1> phi;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 5);
  phi ~ uniform(-1,1);
  sigma ~ normal(3, 3); // Normal with mean 3, st.dev. 3

  for(i in 2:N){
    y[i]~ normal(mu+ phi*(y[i-1] - mu), sigma);
  }
}'
```

```
data <- list(N=N, y=y)
warmup <- 50
niter <- 2000
fit <- stan(model_code=StanModel, data=data, warmup=warmup, iter=niter, chains=4)
```

```
## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.
```

```
## Trying to compile a simple C file
```

```
## Running "C:/PROGRA~1/R/R-44~1.3/bin/x64/Rcmd.exe" SHLIB foo.c
## using C compiler: 'gcc.exe (GCC) 13.3.0'
## gcc -I"C:/PROGRA~1/R/R-44~1.3/include" -DNDEBUG -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/Rcpp/include/" -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/"
-I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/unsupported" -
I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/BH/include" -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/StanHeaders/include/src/" -I"C:/Users/Administrator/AppData/Local/R/win-
library/4.4/StanHeaders/include/" -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppParallel"
DRCPP_PARALLEL_USE_TBB=1 -I"C:/Users/Administrator/AppData/Local/R/win-library/4.4/rstan/include" -
DEIGEN_NO_DEBUG -DBOOST_DISABLE_ASSERTS -DBOOST_PENDING_INTEGER_LOG2_HPP -DSTAN_THREADS -
DUSE_STANC3 -DSTRICT_R_HEADERS -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION -D_HAS_AUTO_PTR_ETC=0 -
include "C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan/math/prim/fun/
std=c++1y -I"C:/rtools44/x86_64-w64-mingw32.static.posix/include" -O2 -Wall -
mfpmath=sse -msse2 -mstackrealign -c foo.c -o foo.o
## cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
## In file included from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen,
## from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen,
## from C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan,
## from <command-line>:
## C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/src/Core/util/Macros.
## 679 | #include <cmath>
## | ~~~~~
## compilation terminated.
## make: *** [C:/PROGRA~1/R/R-44~1.3/etc/x64/Makeconf:289: foo.o] Error 1
```

```
## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.
```

```
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 6.5e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.65 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: WARNING: There aren't enough warmup iterations to fit the
```



```

## Chain 1:      three stages of adaptation as currently configured.
## Chain 1:      Reducing each adaptation stage to 15%/75%/10% of
## Chain 1:      the given number of warmup iterations:
## Chain 1:      init_buffer = 7
## Chain 1:      adapt_window = 38
## Chain 1:      term_buffer = 5
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 1: Iteration:   51 / 2000 [ 2%] (Sampling)
## Chain 1: Iteration:  250 / 2000 [12%] (Sampling)
## Chain 1: Iteration:  450 / 2000 [22%] (Sampling)
## Chain 1: Iteration:  650 / 2000 [32%] (Sampling)
## Chain 1: Iteration:  850 / 2000 [42%] (Sampling)
## Chain 1: Iteration: 1050 / 2000 [52%] (Sampling)
## Chain 1: Iteration: 1250 / 2000 [62%] (Sampling)
## Chain 1: Iteration: 1450 / 2000 [72%] (Sampling)
## Chain 1: Iteration: 1650 / 2000 [82%] (Sampling)
## Chain 1: Iteration: 1850 / 2000 [92%] (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.114 seconds (Warm-up)
## Chain 1:      4.355 seconds (Sampling)
## Chain 1:      4.469 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 2.9e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.29 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: WARNING: There aren't enough warmup iterations to fit the
## Chain 2:      three stages of adaptation as currently configured.
## Chain 2:      Reducing each adaptation stage to 15%/75%/10% of
## Chain 2:      the given number of warmup iterations:
## Chain 2:      init_buffer = 7
## Chain 2:      adapt_window = 38
## Chain 2:      term_buffer = 5
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration:   51 / 2000 [ 2%] (Sampling)
## Chain 2: Iteration:  250 / 2000 [12%] (Sampling)
## Chain 2: Iteration:  450 / 2000 [22%] (Sampling)
## Chain 2: Iteration:  650 / 2000 [32%] (Sampling)
## Chain 2: Iteration:  850 / 2000 [42%] (Sampling)
## Chain 2: Iteration: 1050 / 2000 [52%] (Sampling)
## Chain 2: Iteration: 1250 / 2000 [62%] (Sampling)
## Chain 2: Iteration: 1450 / 2000 [72%] (Sampling)
## Chain 2: Iteration: 1650 / 2000 [82%] (Sampling)
## Chain 2: Iteration: 1850 / 2000 [92%] (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.064 seconds (Warm-up)

```

```

## Chain 2:          4.087 seconds (Sampling)
## Chain 2:          4.151 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 2.9e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.29 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: WARNING: There aren't enough warmup iterations to fit the
## Chain 3:          three stages of adaptation as currently configured.
## Chain 3:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 3:          the given number of warmup iterations:
## Chain 3:          init_buffer = 7
## Chain 3:          adapt_window = 38
## Chain 3:          term_buffer = 5
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration:   51 / 2000 [ 2%] (Sampling)
## Chain 3: Iteration:  250 / 2000 [12%] (Sampling)
## Chain 3: Iteration:  450 / 2000 [22%] (Sampling)
## Chain 3: Iteration:  650 / 2000 [32%] (Sampling)
## Chain 3: Iteration:  850 / 2000 [42%] (Sampling)
## Chain 3: Iteration: 1050 / 2000 [52%] (Sampling)
## Chain 3: Iteration: 1250 / 2000 [62%] (Sampling)
## Chain 3: Iteration: 1450 / 2000 [72%] (Sampling)
## Chain 3: Iteration: 1650 / 2000 [82%] (Sampling)
## Chain 3: Iteration: 1850 / 2000 [92%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.102 seconds (Warm-up)
## Chain 3:          4.291 seconds (Sampling)
## Chain 3:          4.393 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 2.9e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.29 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: WARNING: There aren't enough warmup iterations to fit the
## Chain 4:          three stages of adaptation as currently configured.
## Chain 4:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 4:          the given number of warmup iterations:
## Chain 4:          init_buffer = 7
## Chain 4:          adapt_window = 38
## Chain 4:          term_buffer = 5
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration:   51 / 2000 [ 2%] (Sampling)

```

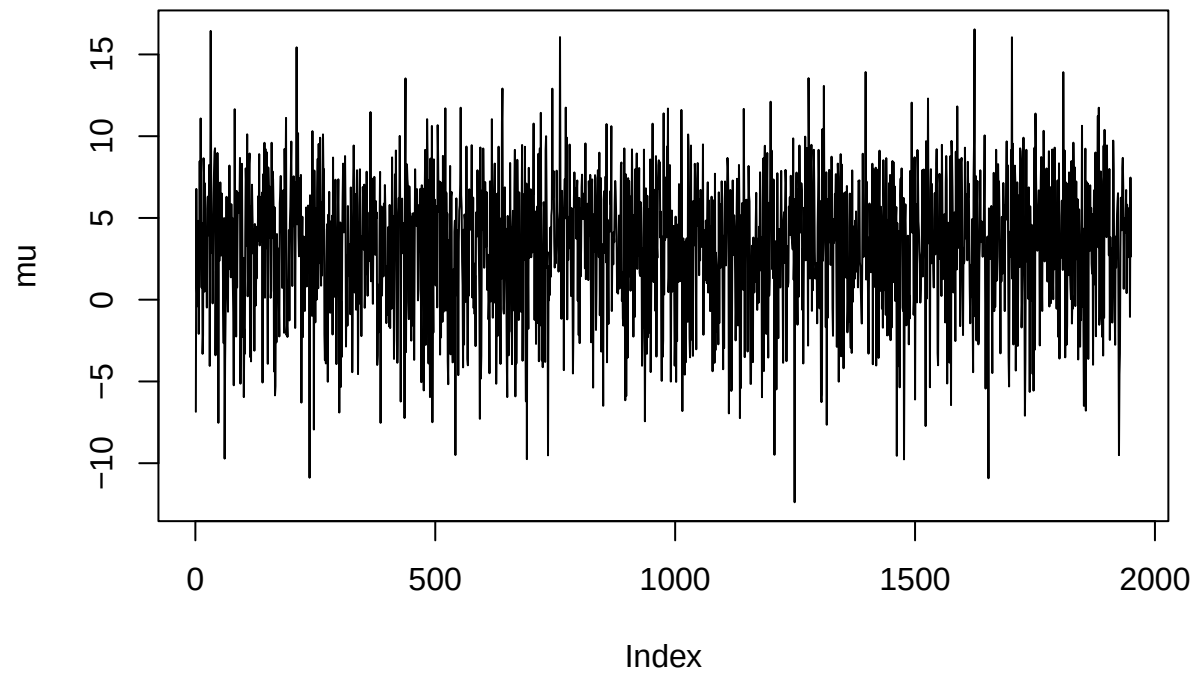
```
## Chain 4: Iteration: 250 / 2000 [ 12%] (Sampling)
## Chain 4: Iteration: 450 / 2000 [ 22%] (Sampling)
## Chain 4: Iteration: 650 / 2000 [ 32%] (Sampling)
## Chain 4: Iteration: 850 / 2000 [ 42%] (Sampling)
## Chain 4: Iteration: 1050 / 2000 [ 52%] (Sampling)
## Chain 4: Iteration: 1250 / 2000 [ 62%] (Sampling)
## Chain 4: Iteration: 1450 / 2000 [ 72%] (Sampling)
## Chain 4: Iteration: 1650 / 2000 [ 82%] (Sampling)
## Chain 4: Iteration: 1850 / 2000 [ 92%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.052 seconds (Warm-up)
## Chain 4: 3.722 seconds (Sampling)
## Chain 4: 3.774 seconds (Total)
## Chain 4:
```

```
# Print the fitted model
print(fit,digits_summary=3)
```

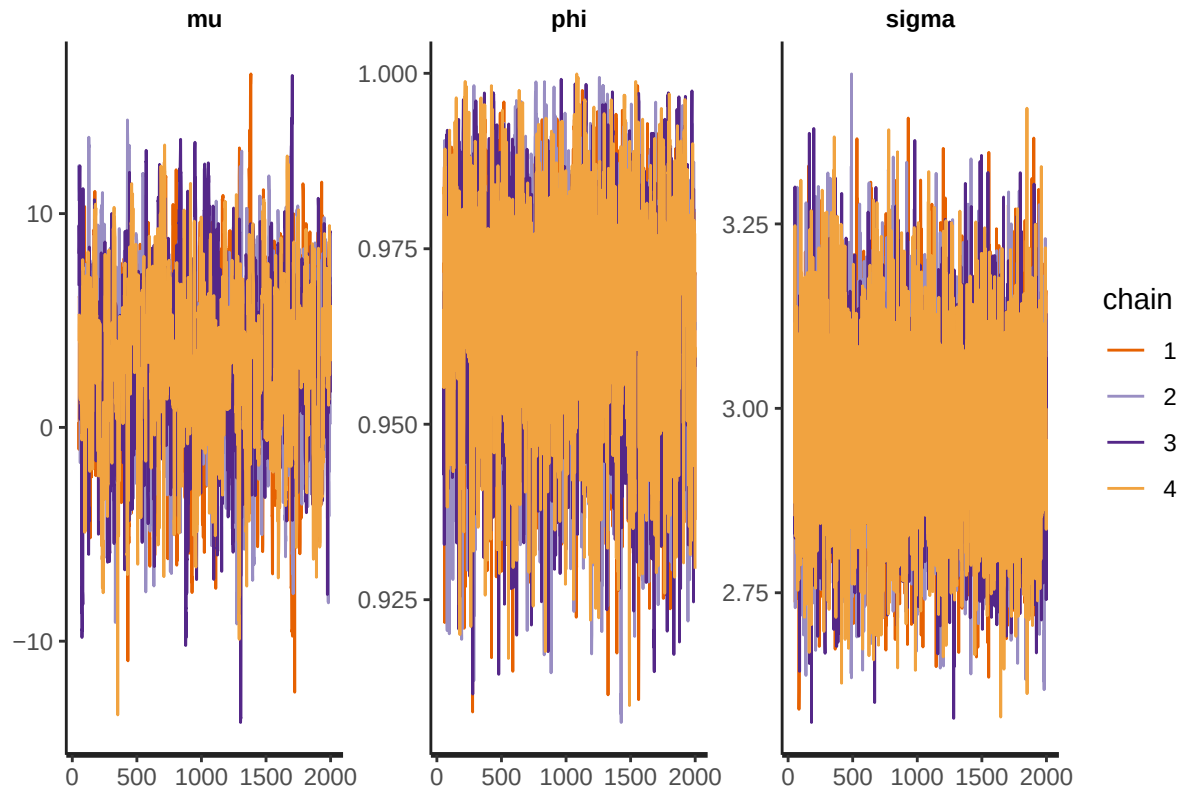
```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=50; thin=1;
## post-warmup draws per chain=1950, total post-warmup draws=7800.
##
##          mean se_mean   sd    2.5%    25%    50%    75%    97.5% n_eff
## mu       3.191   0.135 4.160   -5.583    0.533    3.432    6.053   10.723   951
## phi      0.966   0.000 0.016    0.931    0.955    0.967    0.978    0.994  1896
## sigma    2.963   0.001 0.124    2.739    2.877    2.957    3.044    3.218  7729
## lp__ -476.651   0.039 1.373 -480.080 -477.287 -476.307 -475.630 -475.039  1261
##          Rhat
## mu       1.003
## phi      1.001
## sigma    1.000
## lp__     1.000
##
## Samples were drawn using NUTS(diag_e) at Fri May 16 23:44:52 2025.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
# Extract posterior samples
postDraws <- extract(fit)
# Do traceplots of the first chain
par(mfrow = c(1,1))
plot(postDraws$mu[1:(niter-warmup)],type="l",ylab="mu",main="Traceplot")
```

Traceplot

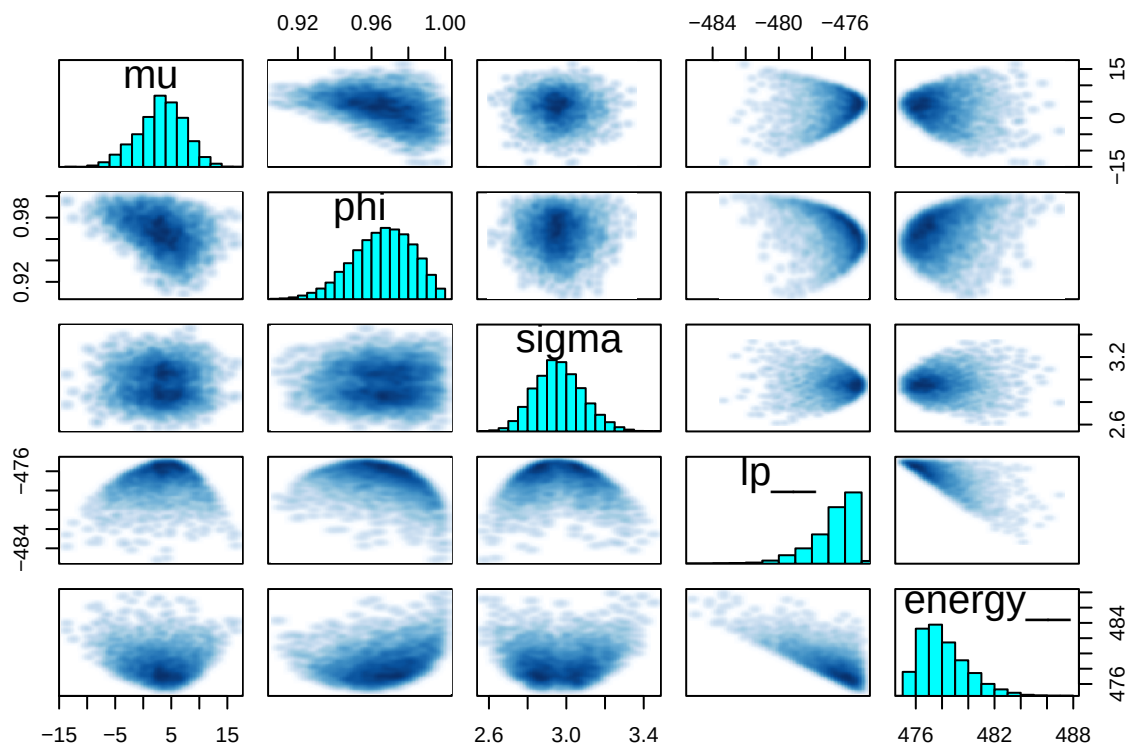


```
# Do automatic traceplots of all chains  
traceplot(fit)
```



```
# Bivariate posterior plots
pairs(fit)
```

```
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
```



```
summary_fit <- summary(fit, pars = c("mu", "phi", "sigma"))$summary
# extract the mean, 95% CI and effective posterior sample
posterior_summary <- summary_fit[, c("mean", "2.5%", "97.5%", "n_eff")]
print(round(posterior_summary, 3))
```

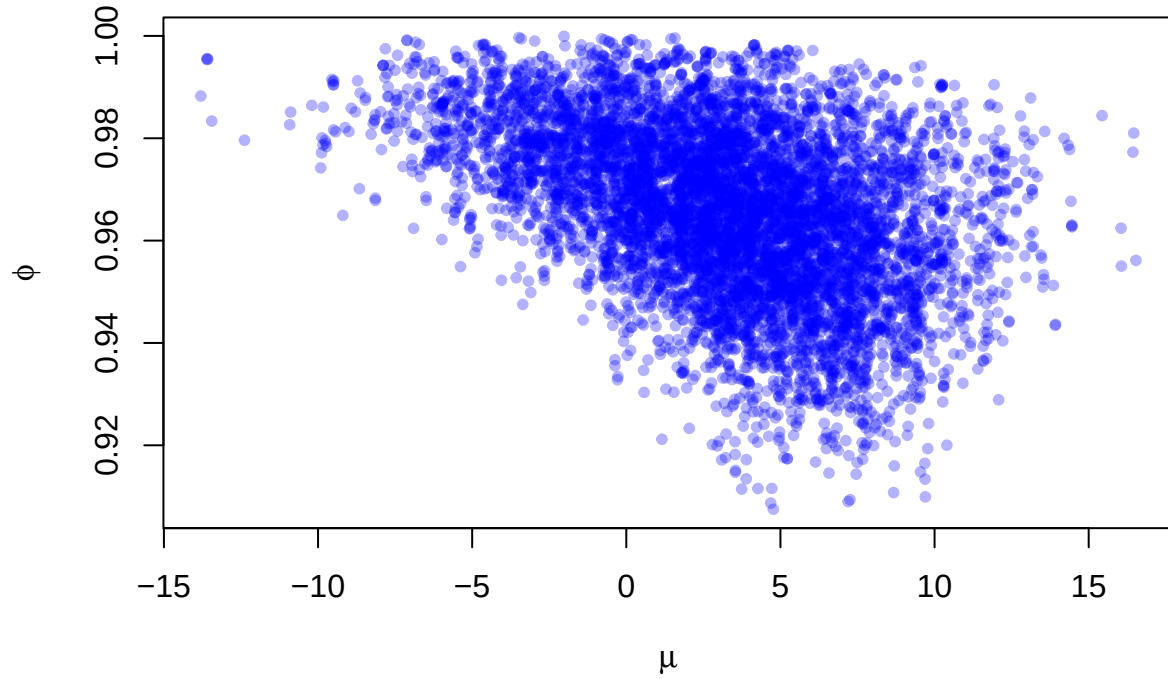
```
##      mean  2.5% 97.5%  n_eff
## mu    3.191 -5.583 10.723  951.075
## phi   0.966  0.931  0.994 1896.228
## sigma 2.963  2.739  3.218 7729.057
```

Mean, 95% CI and effective posterior sample number are shown as above. Despite simulated phi and sigma values are quite close to the true values and the true values lie in the 95% CI, the simulated mean gives too wide 95% CI, making it unable to make a estimate of it. This shows that the data provide little information about μ , and the estimate of μ is not very precise.

```
posterior_samples <- as.data.frame(rstan::extract(fit, pars = c("mu", "phi")))

plot(posterior_samples$mu, posterior_samples$phi,
     xlab = expression(mu), ylab = expression(phi),
     main = "Joint Posterior of mu and phi", pch = 20, col = rgb(0, 0, 1, 0.3))
```

Joint Posterior of μ and ϕ



Both cases have Rhat values < 1.05 , meaning all chains are well-mixed and agree. Besides, the traceplots have shown no stickiness and no drift. Thus, convergence is considered reached for both ϕ values.

When ϕ is equal to 0.4, μ and ϕ is low correlated (cloud-shape plot) and moderate correlated (diagonal elliptical shape) when ϕ is 0.98. This is due to the near-nonstationary behavior of the AR(1) process when ϕ is close to 1. We can conclude that even if convergence is good or acceptable, it does not necessarily mean a good sample for the posterior.